



广东龙芯 2K3000AI 开发板用户手册

修订记录

1

广东龙芯中科电子科技有限公司
广州市黄埔区芯大厦 B3 栋 301 楼龙芯中科 020-22900688



修订记录累积了每次文档更新的说明。最新版本的文档包含以前所有文档版本已更新的内容。

修订日期	版本	修订者	修订说明
2025/11/12	V1.0	广东龙芯技术部	文档创建
2026/4/29	v1.1	广东龙芯技术部	更新 vscode .net electron AI 代码工具安装说明
2026/6/9	v1.2	广东龙芯技术部	更新各软件脚本安装、qt6 安装、lacm、latex、SSH、VNC、RDP 说明



一、开发板简介

1.1 芯片概述

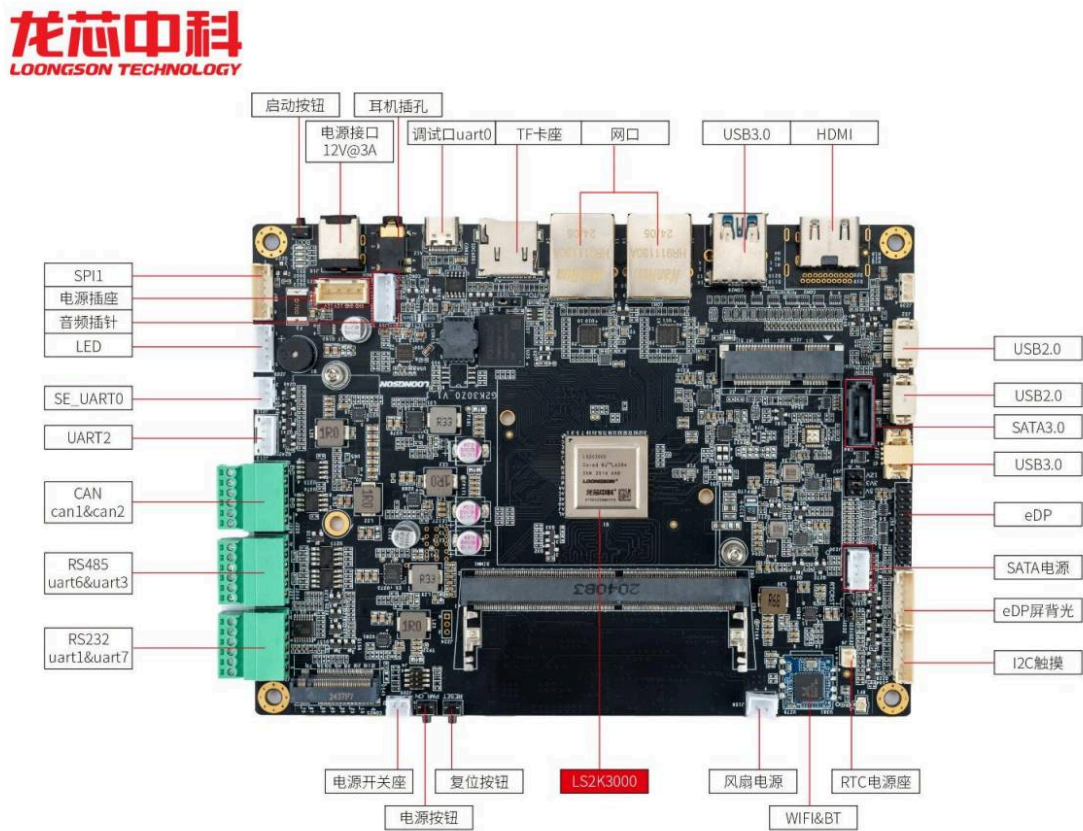
龙芯 2K3000 AI 开发板是基于龙芯 2k3000 处理器的平台产品。2K3000 处理器是基于 LA364E 处理器核的芯片。该芯片集成 8 个 LA364E 处理器核，主频 2.2GHz。该芯片集成第二代自研 GPGPU 核心 LG200，支持通用计算加速和 AI 加速，单精度浮点峰值性能为 256GFLOPS，8 位定点峰值性能为 6.4TOPS。

该芯片集成独立硬件编解码模块，支持各种主流视频格式，支持 eDP/DP/HDMI 三路显示接口输出，分辨率最高可达 4K；集成安全可信模块，可提供安全可信支持和密码服务，在 SM2/3/4 硬件算法模块外，还实现了可供软件编程使用的可重构密码模块；集成丰富的 IO 扩展接口，包括 PCIe3.0、USB3.0/USB2.0、SATA3.0、GMAC、eMMC、SDIO、SPI、LPC、RapidIO 和 CAN-FD 等，满足不同领域的应用需求。

1.2 开发板概述

龙芯 2k3000 AI 开发板显示上支持 HDMI，eDP 独立双显，网络上支持双千兆有线网口，WIFI&BT&4G/5G 网络，板卡配备丰富接口，支持 DDR4 内存（最大 32GB）和 eMMC 存储（最大 128GB）扩展，且具备 -40℃至 85℃宽温运行、抗振动冲击等工业级特性，内置国密安全模块保障数据安全。主要面向工业控制、智能终端、物联网等领域，应用场景如广告机、算力网关、工业计算机等。

二、开发板硬件接口介绍



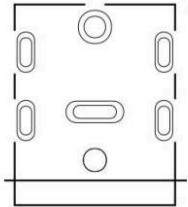
编 号	板载硬件资源	描述
1	CPU	LOONGSON 2K3000 处理器，主频 2.2GHz 64 位 集成 8 个 LA364E 核
3	内存	DDR4 SODIMM 插槽，容量支持 8/16/32GB
4	EMMC	商业级默认容量 32GB， 工业级默认容量 64GB，
5	FLASH	SPI FLASH 容量 8MB
7	DC 电源	DC 电源接口，使用 12V/3A 适配器供电
8	电源插座	
9	USB 接口	2 路 USB3.0 TYPE-A 接口 1 路 USB3.0 接口（插针） 3 路 USB2.0 接口（插针）
10	HDMI 输出	1 个标准接口
11	eDP 接口	可直接驱动 eDP 接口液晶屏，可支持 USB/I2C 接口触摸屏



12	TYPE-C 接口	1 个 TYPE-C 接口，用于串口调试
13	耳机输出	支持一路三/四段耳机插入
14	音频输出	支持喇叭输出，可外接功放扩展板
15	其他接口	2 路 CAN-FD、2 路 RS232、2 路 RS485、1 路 UART TTL 接口
16	TP 接口	1 个，可接 I2C 接口的 TP 屏
17	SATA 接口	1 路 SATA 接口的 SATA 硬盘供电接口
18	NVME 接口（PCI-E x4）	支持 NVME SSD 硬盘
19	SPI 接口	1 路 SPI 接口
20	网口	2 路千兆网口，支持 10/100/1000Mbps 自适应
21	WIFI&BT(USB2.0)	板载 WIFI&BT 模组
22	TF 卡	1 路 TF 卡接口
23	RTC 电源座	接 RTC 纽扣电池
24	风扇电源接口	接 CPU 散热风扇
25	MINI PCIE 接口	支持 4G/5G 模块，兼容 USB/PCIE X1 接口，板载 SIM 卡插槽
26	LED 指示灯	
27	复位按钮	重启系统
28	电源按钮	开机
29	启动按钮	开机后可当普通按钮使用

2.1 电源接口

DC 电源接口，使用 12V 3A 适配器供电，直接连接**配套电源适配器**使用。如果使用的电源适配器功率过小，会导致无法正常开机。





2.2 内存资源

开发板集成 SODIMM 插槽，支持 8GB /16GB/32GB DDR4 内存，注意，开发板默认不带内存条，需要自备。不接内存条，开发板是无法开机的。



2.3 存储资源

开发板支持 NVME/SATA/EMMC 等存储接口，默认板载 EMMC 容量为 32GB。SSD 可使用 NVME 接口或 SATA 接口的硬盘（需自备）。

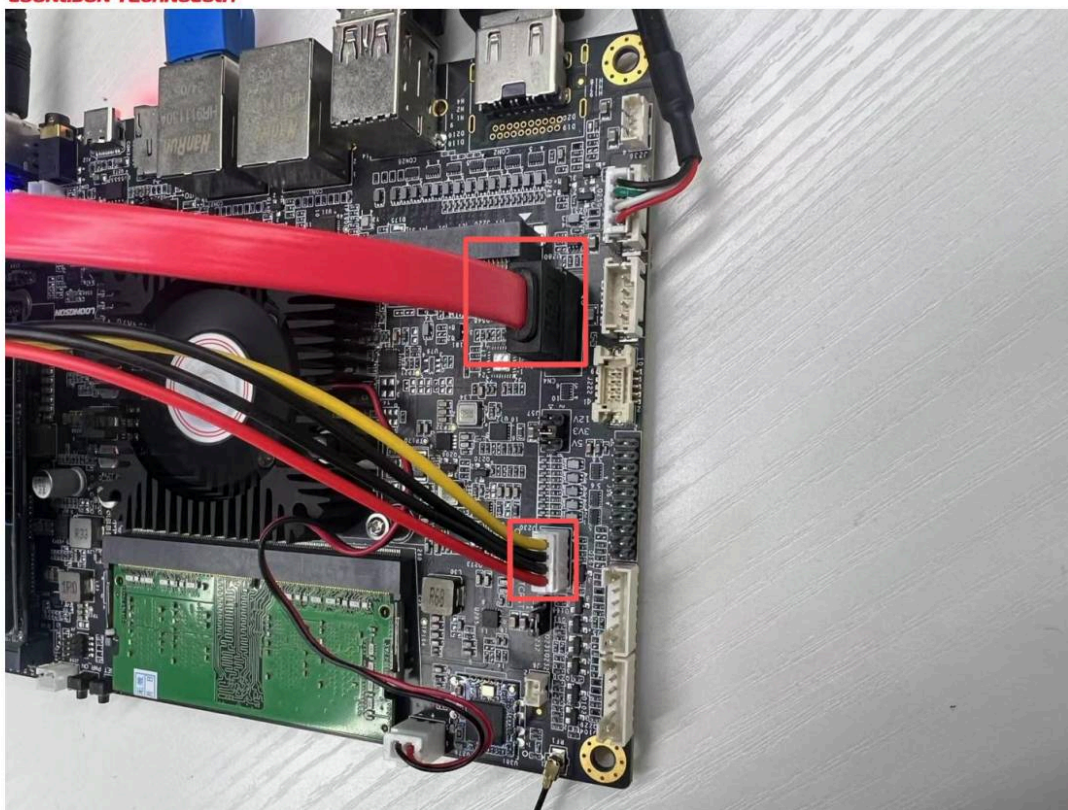
龙芯中科
LOONGSON TECHNOLOGY



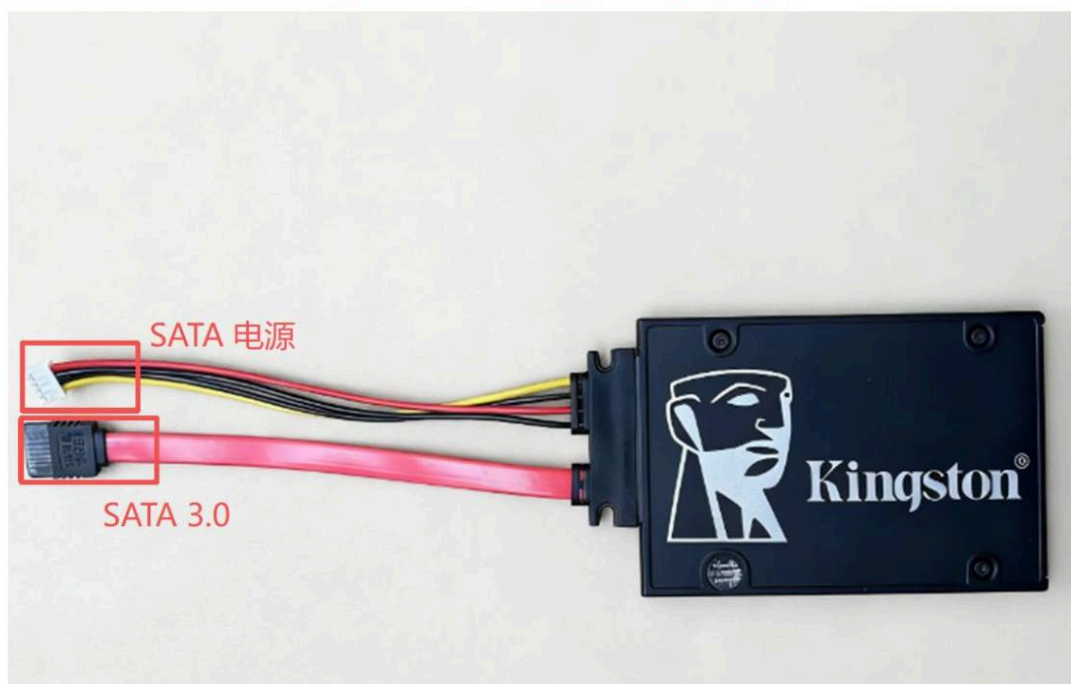
广东龙芯 2k3000 AI 评估板存储接口说明图

如果使用 NVME 接口，则将硬盘对准卡槽接好，拧上螺丝即可。如果使用 SATA 接口接外置硬盘，则将 SATA3.0 和 SATA 电源分别对准接好。卡槽有固定方向，方向一致才能成功接入。使用 SATA 接口接外置硬盘如下图所示：

龙芯中科
LOONGSON TECHNOLOGY



广东龙芯 2k3000 AI 评估板 SATA 接口接线图



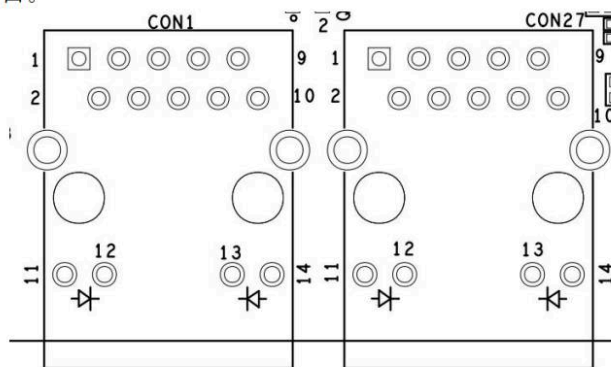
SATA 接口的 SSD 示意图



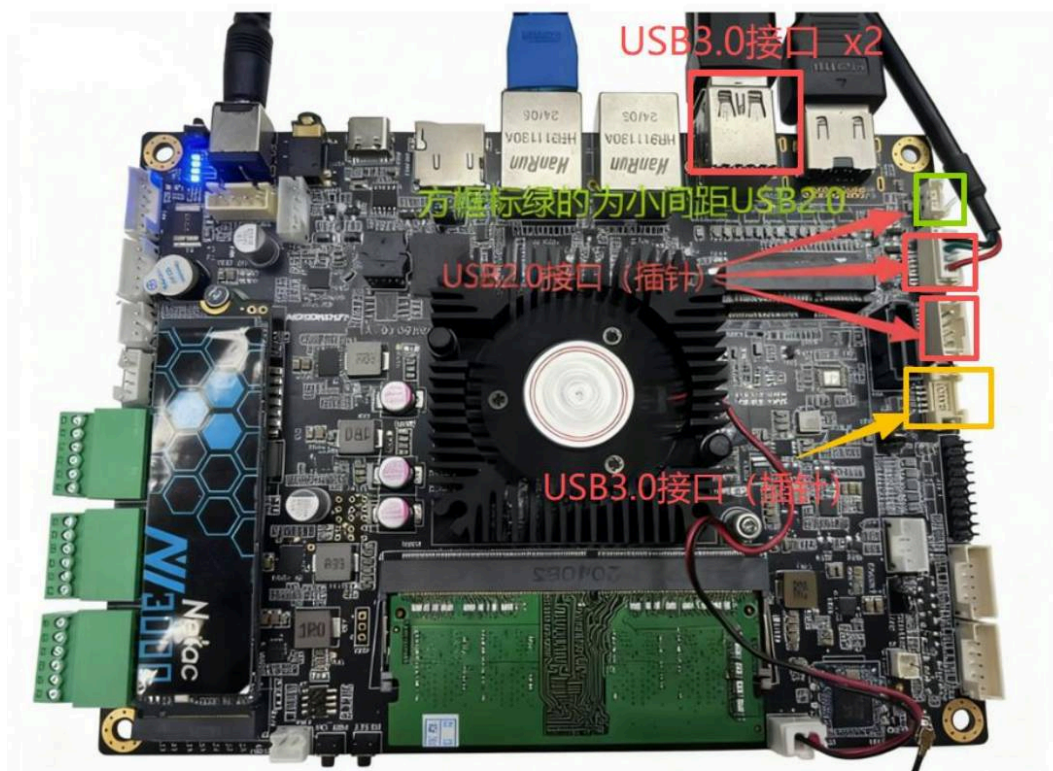
SATA 转接线购买链接可参考：

2.4 网口

集成两个千兆以太网口。



2.5 USB 接口

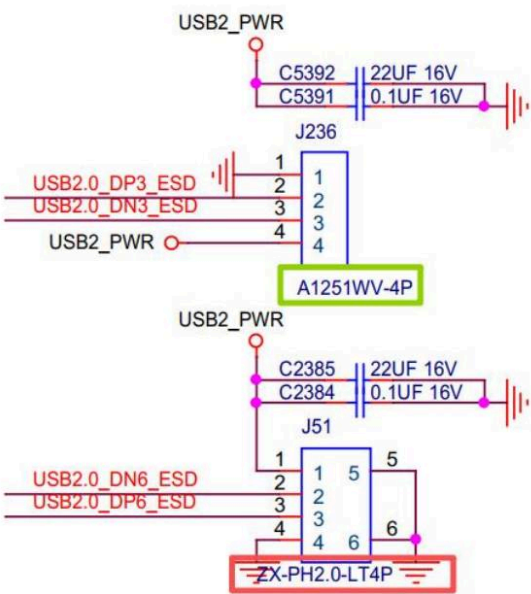




2 路 USB3.0 TYPE-A 接口、1 路 USB3.0 接口（插针）、3 路 USB2.0 接口（插针）带有插针的接口。开发板提供一个双层 USB3.0 TYPE-A 接口，用于开发板连接鼠标、键盘和 U 盘等 USB 外设，对于带有插针的接口，可根据需要外接 USB 扩展线。

注意！！USB3.0 接口（插针）为外设专用接口，默认不配外接 USB3.0 扩展线。如需使用，咨询原厂技术支持。

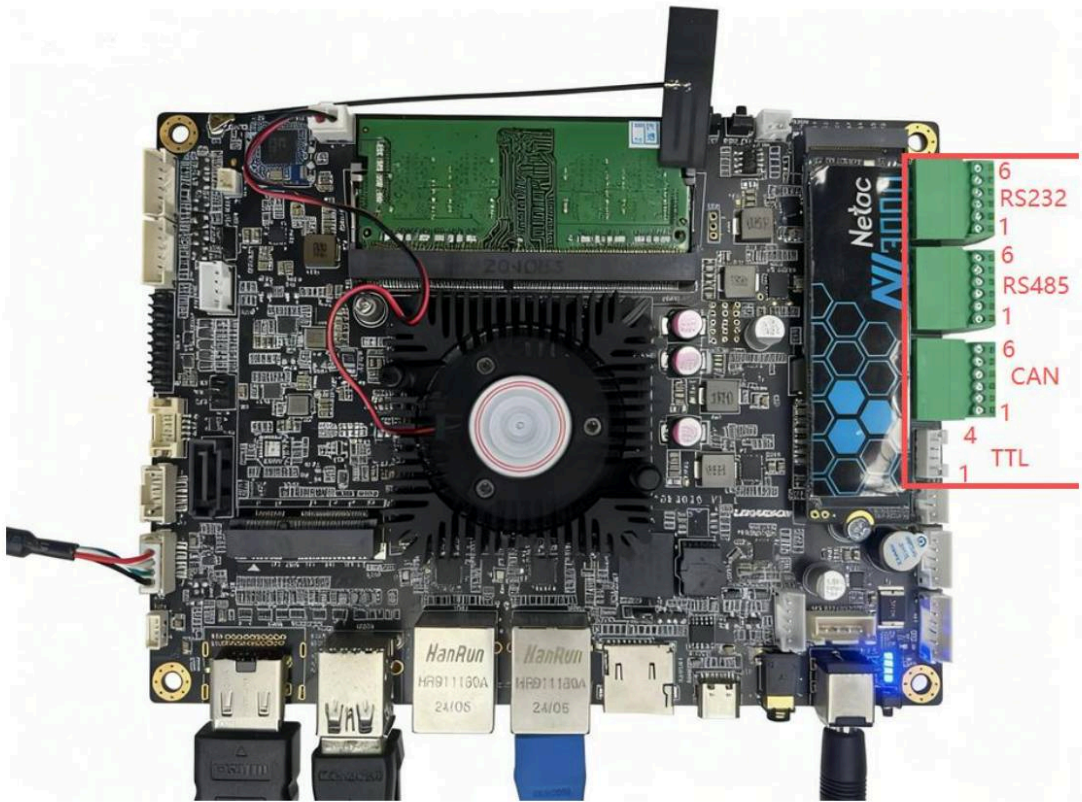
绿色方框中的 USB2.0 接口插座采用 A1251WV-4P，间距 1.25mm。红色方框中的 USB2.0 接口插座采用 ZX-PH2.0-LT4P，间距 2mm。两者使用的扩展线不一致，可以通过原理图找到 USB2.0 接口相关信息，如下图所示。



USB2.0 接口（插针）原理图

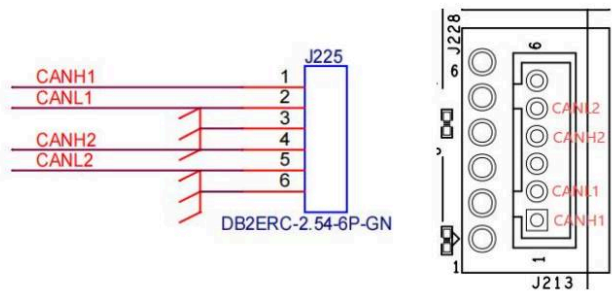


2. 6 CAN/RS485/RS232/TTL/调试接口



2. 6. 1 CAN 接口

开发板支持 2 路 CAN 接口。CAN 接口的插座引脚定义、引脚对应开发板接口位置如下图。与 TF 卡为复用关系，只能二选一。

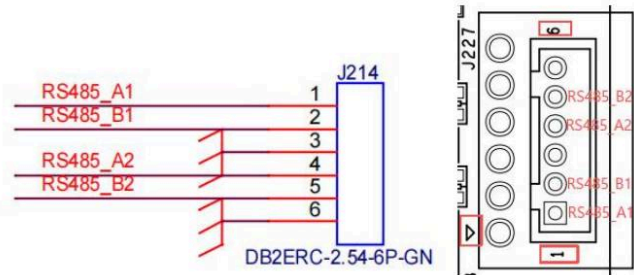


2. 6. 2 RS485 接口

开发板支持两路 RS485 接口。RS485 接口的插座引脚定义、引脚对应开发板接口位置如下图。



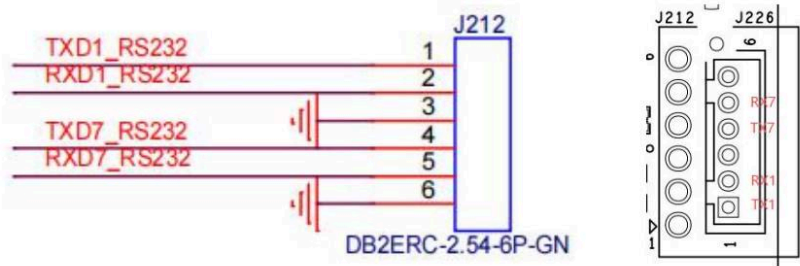
RS485_1 对应 uart6
RS485_2 对应 uart3



2.6.3 RS232 接口

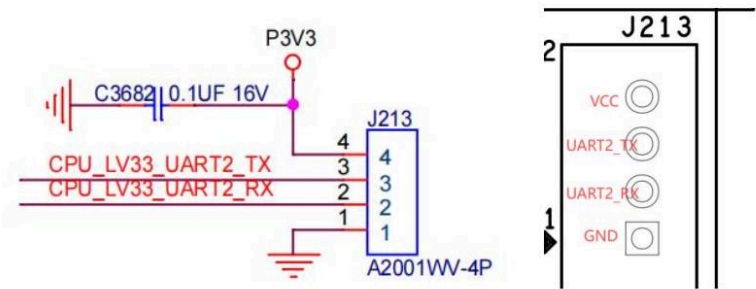
开发板支持两路 RS232 接口。RS232 接口的插座引脚定义、引脚对应开发板接口位置如下图。

RS232_1 对应 uart1
RS232_2 对应 uart7



2.6.4 TTL 接口

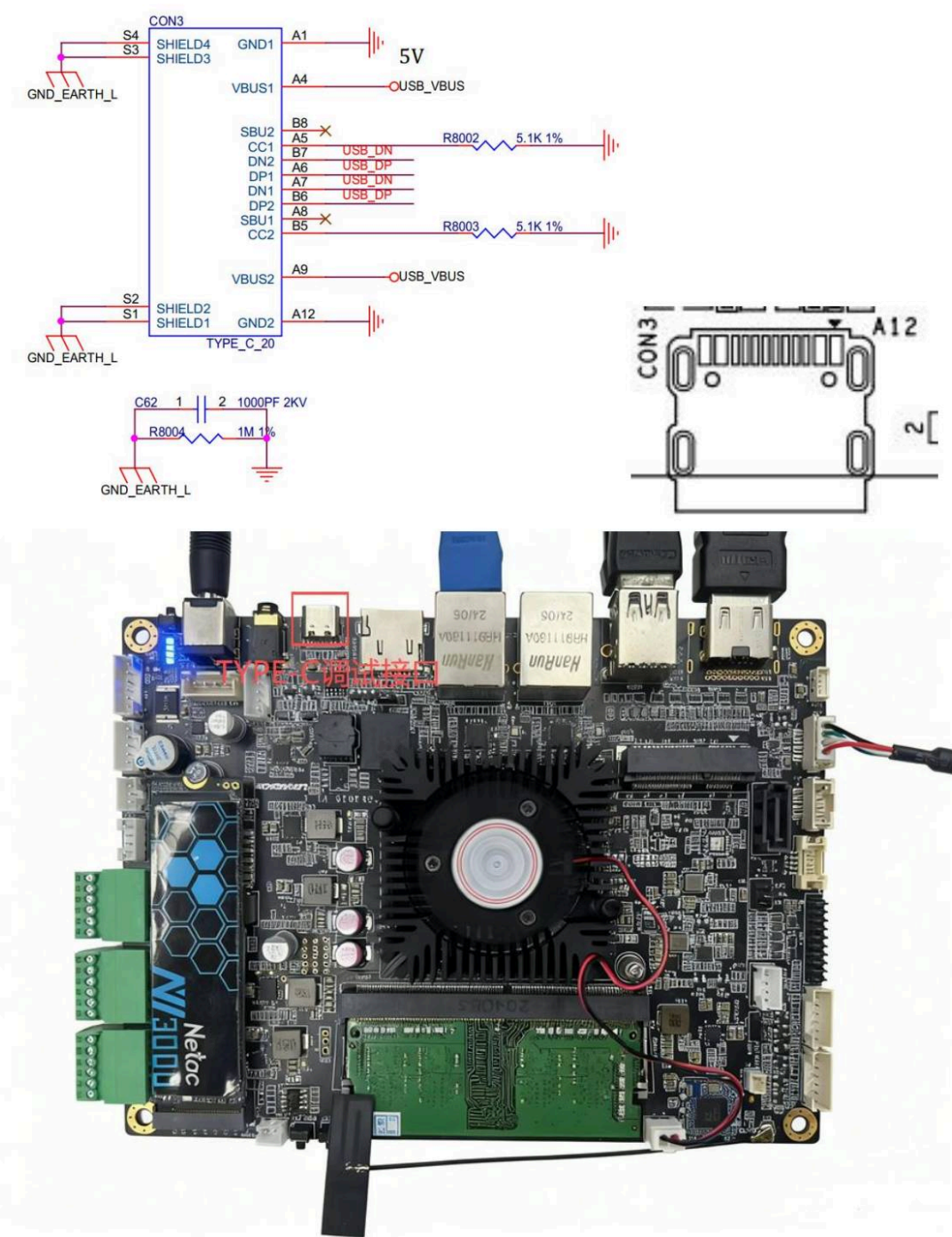
开发板支持一路 UART_TTL 调试接口。UART_TTL 插座接口的引脚定义、引脚对应开发板接口位置如下图。UART_TTL 对应 uart2。





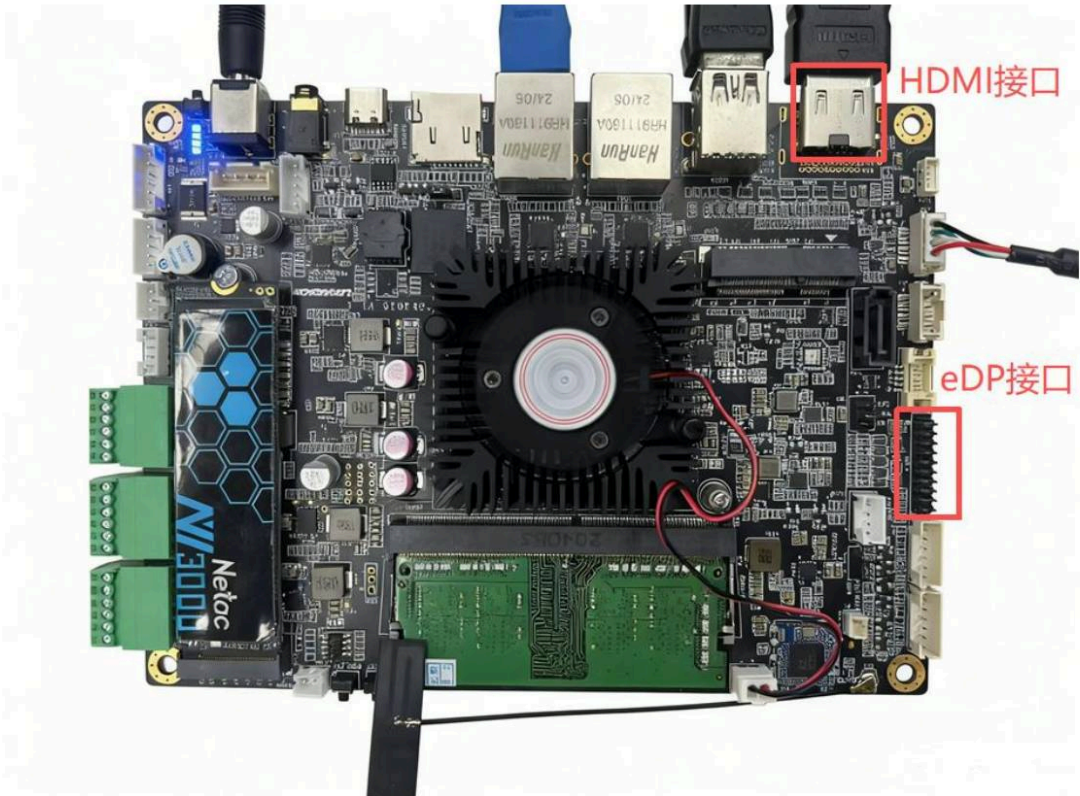
2.6.5 TYPE-C 调试接口

开发板支持一路 TYPE-C 调试。对应设备为 ttyS0，默认波特率为 115200。TYPE-C 调试接口的引脚定义、引脚对应开发板接口位置如下图。UART_TTL 对应 uart0。

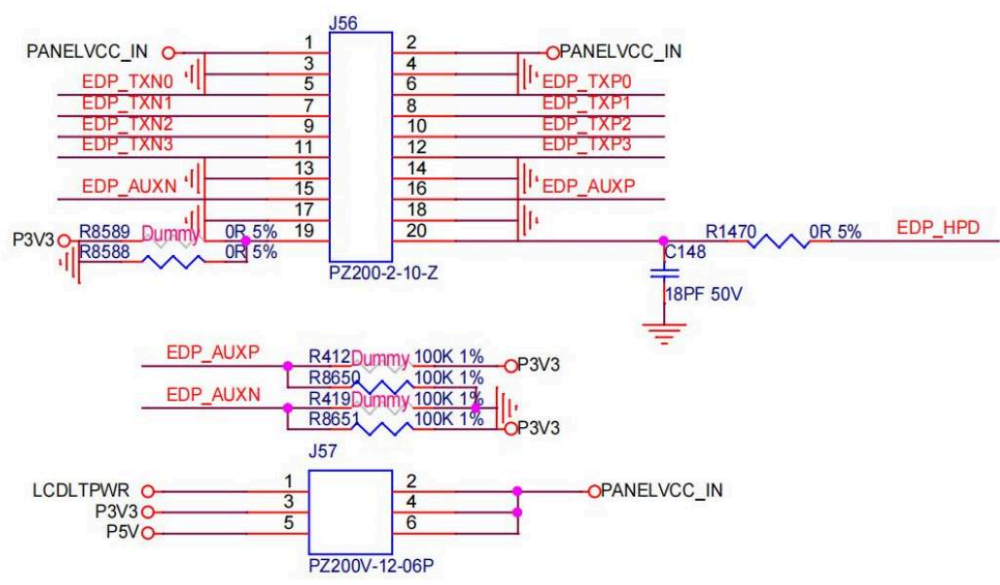




2.7 显示接口



支持 1 路 HDMI，1 路 eDP 独立双显。





eDP 接口的引脚定义和电源接口的如上图。

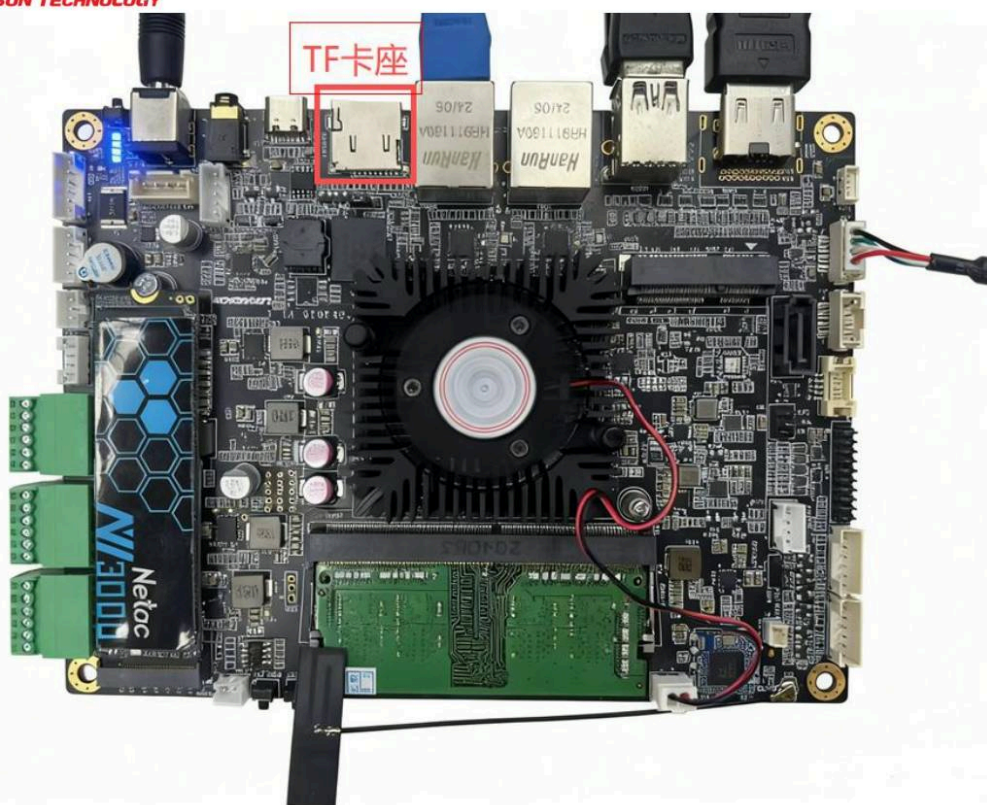
2.8 天线接口



2.9 TF 卡座

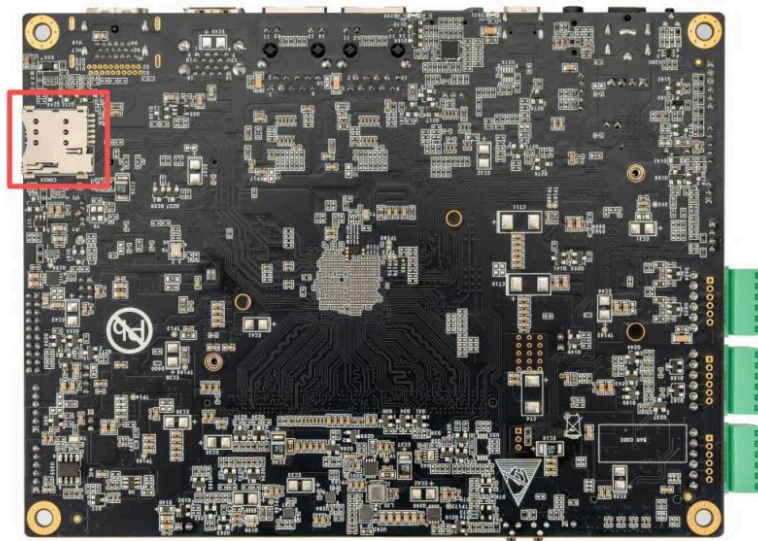
开发板包含了一个 TF 卡接口，以提供用户访问 TF 卡存储器，用于存储操作系统镜像以及其他用户数据文件。与 TF 卡为复用关系，只能二选一。

龙芯中科
LOONGSON TECHNOLOGY



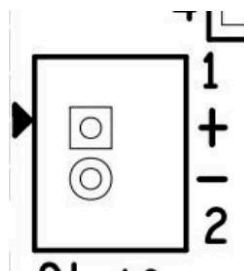
2.10 SIM 卡槽

开发板提供了一个可以插入电话卡的 Nano SIM 卡槽，位于开发板背面。

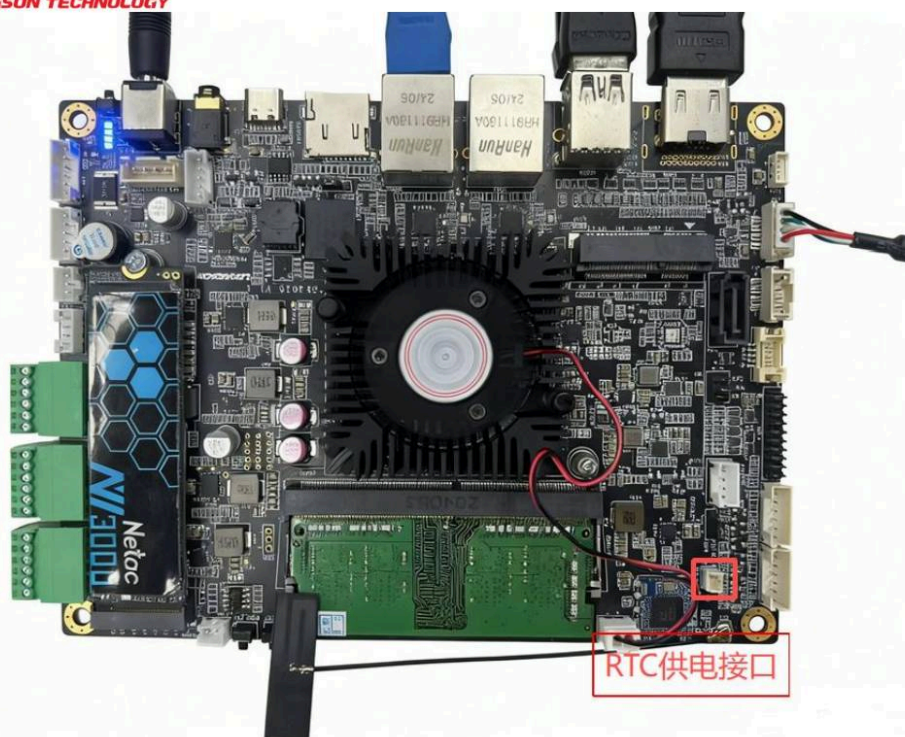


2.10 RTC 供电接口

开发板中提供了一个 RTC 实时时钟的供电接口。LS2K3000 芯片中集成了 RTC 功能，计时精确到 0.1 秒。为了产品掉电以后，实时时钟还可以正常运行，一般需要另外配一个电池给时钟，芯片供电，该接口为电池座，搭配规格为 CR2032 的纽扣电池使用。接口信号连接至芯片中的 VBAT 引脚。

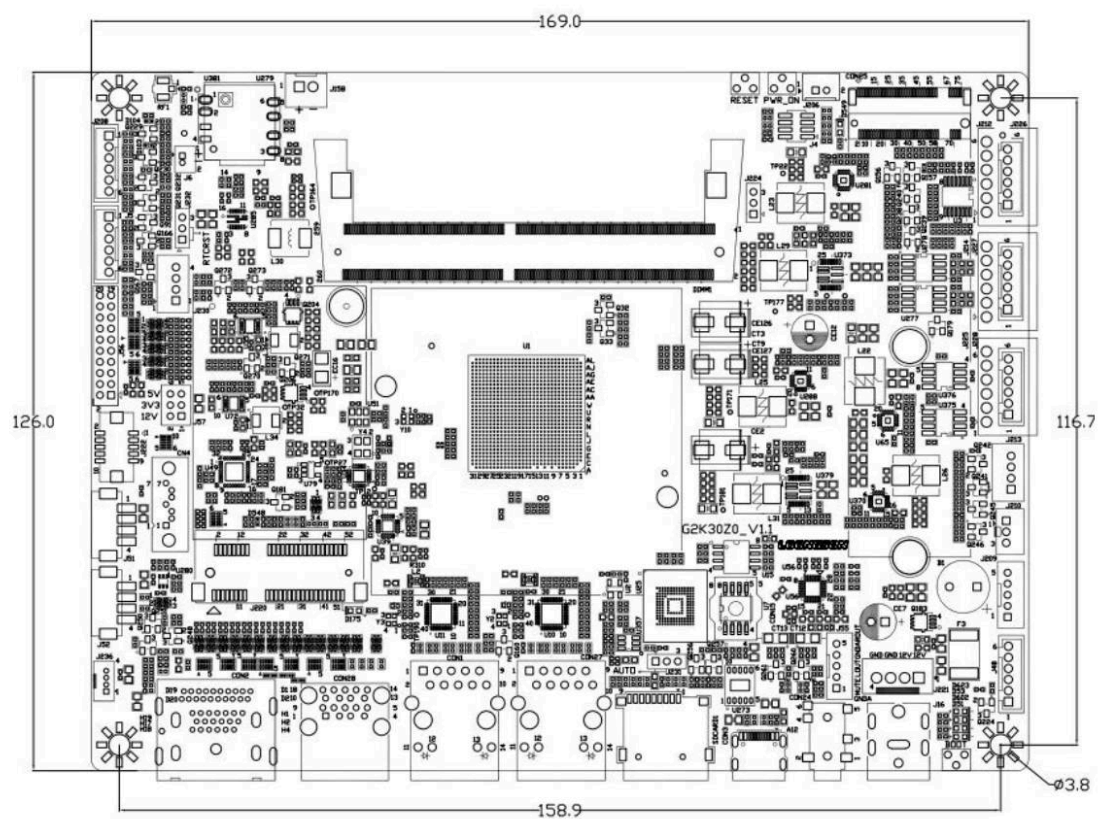


龙芯中科
LOONGSON TECHNOLOGY



2.11 板卡尺寸

尺寸大小 169mm*126mm



广东龙芯 2k3000 AI 评估板尺寸图



2. 12 芯片引脚复用关系

名称	功能 2	功能 3	功能 4	功能 5	功能 6	功能 7	说明
DDR_A00	LPDDR0_A0						需按组配置复用
DDR_A01	LPDDR0_A1						
DDR_A02	LPDDR0_A2						
DDR_A03	LPDDR0_A3						
DDR_A04	LPDDR0_A4						
DDR_A05	LPDDR0_A5						
DDR_A06	LPDDR1_A0						
DDR_A07	LPDDR1_A1						
DDR_A08	LPDDR1_A2						
DDR_A09	LPDDR1_A3						
DDR_A10	LPDDR1_A4						
DDR_A11	LPDDR1_A5						
DDR_A12	LPDDR1_CS0n						
DDR_A13	LPDDR1_CS1n						
DDR_ACTn							

DDR_CASn	LPDDR1_CKE1					
DDR_CKE0	LPDDR0_CKE0					
DDR_CKE1	LPDDR0_CKE1					
DDR_CKN0	LPDDR0_CKN					
DDR_CKN1	LPDDR1_CKN					
DDR_CKP0	LPDDR0_CKP					
DDR_CKP1	LPDDR1_CKP					
DDR_ODT0	LPDDR_ODT0					
DDR_ODT1	LPDDR_ODT1					
DDR_RASn						
DDR_RESETEn	LPDDR_RESETEn					
DDR_SCS0n	LPDDR0_CS0n					
DDR_SCS1n	LPDDR0_CS1n					
DDR_WEn	LPDDR1_CKE0					
DDR_DM0n	LPDDR0_DM10					
DDR_DM1n	LPDDR0_DM11					
DDR_DM2n	LPDDR0_DM12					
DDR_DM3n	LPDDR0_DM13					



DDR_DM4n	LPDDR1_DMI0					
DDR_DM5n	LPDDR1_DMI1					
DDR_DM6n	LPDDR1_DMI2					
DDR_DM7n	LPDDR1_DMI3					
DDR_DQ00	LPDDR0_DQ00					
DDR_DQ01	LPDDR0_DQ01					
DDR_DQ02	LPDDR0_DQ02					
DDR_DQ03	LPDDR0_DQ03					
DDR_DQ04	LPDDR0_DQ04					
DDR_DQ05	LPDDR0_DQ05					
DDR_DQ06	LPDDR0_DQ06					
DDR_DQ07	LPDDR0_DQ07					
DDR_DQ08	LPDDR0_DQ08					
DDR_DQ09	LPDDR0_DQ09					
DDR_DQ10	LPDDR0_DQ10					
DDR_DQ11	LPDDR0_DQ11					
DDR_DQ12	LPDDR0_DQ12					
DDR_DQ13	LPDDR0_DQ13					

DDR_DQ14	LPDDR0_DQ14					
DDR_DQ15	LPDDR0_DQ15					
DDR_DQ16	LPDDR0_DQ16					
DDR_DQ17	LPDDR0_DQ17					
DDR_DQ18	LPDDR0_DQ18					
DDR_DQ19	LPDDR0_DQ19					
DDR_DQ20	LPDDR0_DQ20					
DDR_DQ21	LPDDR0_DQ21					
DDR_DQ22	LPDDR0_DQ22					
DDR_DQ23	LPDDR0_DQ23					
DDR_DQ24	LPDDR0_DQ24					
DDR_DQ25	LPDDR0_DQ25					
DDR_DQ26	LPDDR0_DQ26					
DDR_DQ27	LPDDR0_DQ27					
DDR_DQ28	LPDDR0_DQ28					
DDR_DQ29	LPDDR0_DQ29					
DDR_DQ30	LPDDR0_DQ30					
DDR_DQ31	LPDDR0_DQ31					



DDR_DQ32	LPDDR1_DQ00					
DDR_DQ33	LPDDR1_DQ01					
DDR_DQ34	LPDDR1_DQ02					
DDR_DQ35	LPDDR1_DQ03					
DDR_DQ36	LPDDR1_DQ04					
DDR_DQ37	LPDDR1_DQ05					
DDR_DQ38	LPDDR1_DQ06					
DDR_DQ39	LPDDR1_DQ07					
DDR_DQ40	LPDDR1_DQ08					
DDR_DQ41	LPDDR1_DQ09					
DDR_DQ42	LPDDR1_DQ10					
DDR_DQ43	LPDDR1_DQ11					
DDR_DQ44	LPDDR1_DQ12					
DDR_DQ45	LPDDR1_DQ13					
DDR_DQ46	LPDDR1_DQ14					
DDR_DQ47	LPDDR1_DQ15					
DDR_DQ48	LPDDR1_DQ16					
DDR_DQ49	LPDDR1_DQ17					

DDR_DQ50	LPDDR1_DQ18					
DDR_DQ51	LPDDR1_DQ19					
DDR_DQ52	LPDDR1_DQ20					
DDR_DQ53	LPDDR1_DQ21					
DDR_DQ54	LPDDR1_DQ22					
DDR_DQ55	LPDDR1_DQ23					
DDR_DQ56	LPDDR1_DQ24					
DDR_DQ57	LPDDR1_DQ25					
DDR_DQ58	LPDDR1_DQ26					
DDR_DQ59	LPDDR1_DQ27					
DDR_DQ60	LPDDR1_DQ28					
DDR_DQ61	LPDDR1_DQ29					
DDR_DQ62	LPDDR1_DQ30					
DDR_DQ63	LPDDR1_DQ31					
DDR_DQSN0	LPDDR0_DQSN0					
DDR_DQSN1	LPDDR0_DQSN1					
DDR_DQSN2	LPDDR0_DQSN2					
DDR_DQSN3	LPDDR0_DQSN3					



LPC_LFRAMEn	LIO_AD13					GP1061	
LPC_RESETEn	LIO_AD14					GP1062	
LPC_SIRQ	LIO_AD15					GP1063	
PWM0	LIO_CS00					GP1050	可单独配置复用
PWM1	LIO_CS01					GP1051	可单独配置复用
PWM2	LIO_AD07					GP1052	可单独配置复用
SDIO_CLK	LIO_A00		CAN0_TX	UART3_TX			SDIO和LIO不可与其他功能同时用；
SDIO_CMD	LIO_A01		CAN0_RX	UART3_RX			CAN0与UART3可单独配置复用；
SDIO_DATA0	LIO_A02		CAN1_TX	UART4_TX			CAN1与UART4可单独配置复用；
SDIO_DATA1	LIO_A03		CAN1_RX	UART4_RX			CAN2与UART5可单独配置复用；
SDIO_DATA2	LIO_A04		CAN2_TX	UART5_TX			
SDIO_DATA3	LIO_A05		CAN2_RX	UART5_RX			
SPI0_HOLDn	SPI0_CS3n						
SPI0_WPn	SPI0_CS2n						
SPI1_CSn	LIO_CS02	SDI01_D3	PWM3				SPI1/LIO/SDI01不可与其他功能同时用；
SPI1_HOLDn	LIO_RDY	SDI01_D2	I2C2_SDA	SPI1_CS3n	UART7_RX		PWM3/I2C2/AVS可同时使用；
SPI1_SCK	LIO_CS03	SDI01_CLK	AVS_CLK				PWM3/UART7/AVS可同
SPI1_SDI	LIO_ADLOCK	SDI01_D0	AVS_MDATA				

SPI1_SDO	LIO_WRn	SDI01_CMD	AVS_SDATA				时使用；
SPI1_WPn	LIO_RDn	SDI01_D1	I2C2_SCL	SPI1_CS2n	UART7_TX		
CHIP_CONFIG6						GP1046	复位阶段为输入，作为配置信号；复位撤销后可作为GPIO
CHIP_CONFIG7						GP1047	
PROCHOTn						GP1049	可单独配置复用
SATA_LEDn						GP1053	可单独配置复用
THERMTRIPn						GP1048	可单独配置复用
UART1_CTS	UART6_RX					GP1032	
UART1_RTS	UART6_TX					GP1031	UART1 双线模式下可与UART6 同时使用
UART1_RXD						GP1030	UART6 可与GPIO[30:29]同时使用
UART1_TXD						GP1029	
RGMI11_MDCK						GP104	
RGMI11_MDIO						GP105	
RGMI11_RCLK						GP107	
RGMI11_RCTL						GP106	需按组配置复用
RGMI11_RXD0						GP108	
RGMI11_RXD1						GP109	
RGMI11_RXD2						GP1010	



RGMII1_RXD3						GPIO11	
RGMII1_TCLK						GPIO13	
RGMII1_TCTL						GPIO12	
RGMII1_TXD0						GPIO14	
RGMII1_TXD1						GPIO15	
RGMII1_TXD2						GPIO16	
RGMII1_TXD3						GPIO17	
USB3_RXN3	SATA_RXN3						需按组配置复用
USB3_RXP3	SATA_RXP3						
USB3_TXN3	SATA_TXN3						
USB3_TXP3	SATA_TXP3						
USB_OC0						GPIO54	可单独配置复用
USB_OC1						GPIO55	可单独配置复用

DDR_DQSN4	LPDDR1_DQSN0						
DDR_DQSN5	LPDDR1_DQSN1						
DDR_DQSN6	LPDDR1_DQSN2						
DDR_DQSN7	LPDDR1_DQSN3						
DDR_DQSP0	LPDDR0_DQSP0						
DDR_DQSP1	LPDDR0_DQSP1						
DDR_DQSP2	LPDDR0_DQSP2						
DDR_DQSP3	LPDDR0_DQSP3						
DDR_DQSP4	LPDDR1_DQSP0						
DDR_DQSP5	LPDDR1_DQSP1						
DDR_DQSP6	LPDDR1_DQSP2						
DDR_DQSP7	LPDDR1_DQSP3						
EMMC_CLK						GPIO18	需按组配置复用
EMMC_CMD						GPIO19	
EMMC_DATA0						GPIO20	
EMMC_DATA1						GPIO21	
EMMC_DATA2						GPIO22	
EMMC_DATA3						GPIO23	

三、开发板使用

开发板在线资料：<https://gitcode.com/open-loongarch/docs-2k3000>

3.1 接线说明

接线时，务必保证完成正确接线后再上电。**装好内存条再上电！**
供电使用 DC 12V 电源适配器，插入电源接口。**注意：如果电源适配器电压过低，会导致供电不足，系统异常。**
显示使用 HDMI 则将 HDMI 线两端接到显示屏和开发板对应的 HDMI 接口上。如果使用 eDP 接口，



则将 eDP 排线对准 J56 插座，插座左上带有小三角形为引脚 1。

3.2 系统默认账号密码、版本、固件信息

账号: loongson

密码: qwer1234

查看系统版本: `cat /etc/loongnix_version`

```
ls@ls-pc:~$ cat /etc/loongnix_version
SystemName=Loongnix
ProductType=Desktop
VersionID=20
Release=7
MinorVersion=RC2
BuildTime=20251030
```

查看固件版本 `cat /sys/firmware/loongson/boardinfo`

```
ls25@ls25-pc:~$ cat /sys/firmware/loongson/boardinfo
BIOS Information
Vendor           : Loongson
Version          : Loongson-UDK2018-V4.0.05878-stable202411-363143b6
ROM Size        : 8192 KB
Release Date     : 07/24/25 20:22:31

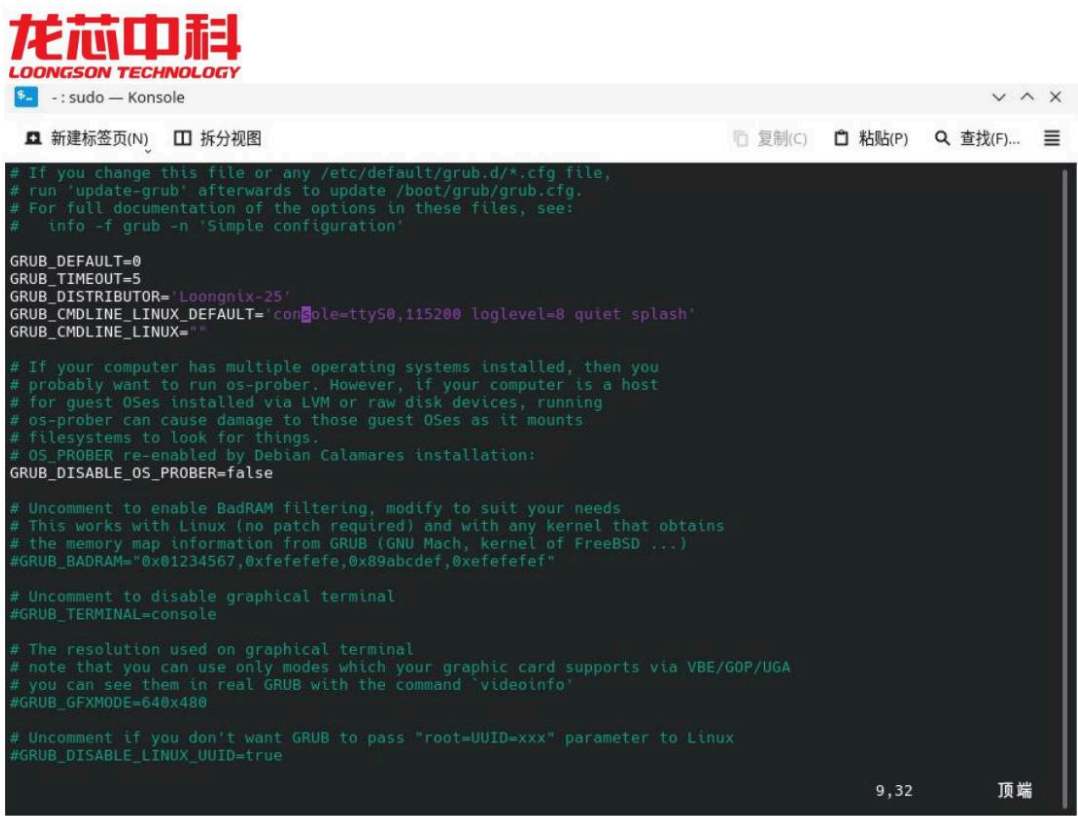
Board Information
Manufacturer     : Loongson
Board Name       : EVB
Family           : LOONGSON64
```

3.3 配置如何支持从串口输出

`sudo vim /etc/default/grub`

在 GRUB_CMDLINE_LINUX_DEFAULT 上添加 `console=ttyS0,115200 loglevel=8`

之后执行 `sudo update-grub2` 之后重启即可。



3.4 使用“龙芯应用合作社”安装软件

通过开始菜单找到“龙芯应用合作社”，搜索想要安装的软件，点击安装即可。以安装 QQ 为例，安装完成后会在桌面创建快捷方式，打开就可以使用了。





3.5 如何安装 Vscode

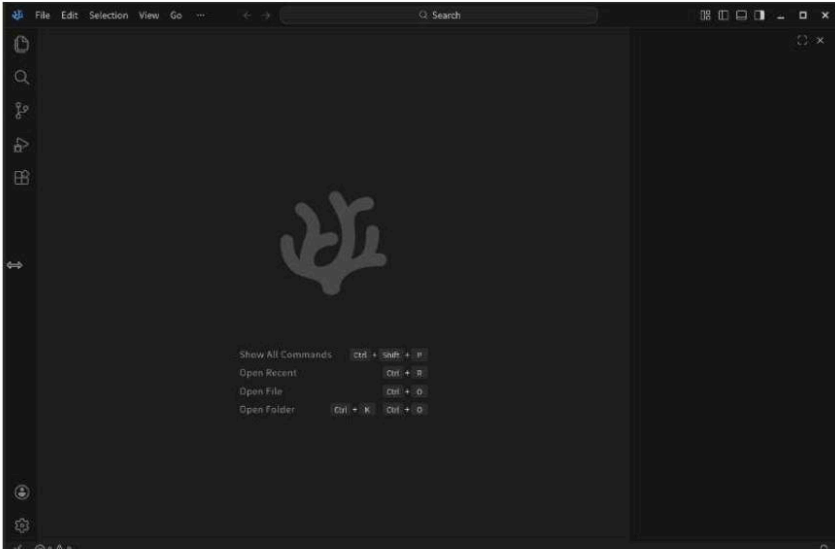
VSCodeium 是社区对 Code-OSS 的一个开源封装版本，提供了一套完全开源的解决方案，开箱即用的方案，其中包括开源的插件仓库（<https://open-vsx.org/>），能够直接下载扩展插件。

可通过以下链接下载 VSCodeium：

<https://github.com/VSCodeium/vscodium/releases/tag/1.112.01907>

Loong 64bits		
Linux	.tar.gz	VSCodeium-linux-loong64-1.112.01907.tar.gz
	Remote Host	vscodium-reh-linux-loong64-1.112.01907.tar.gz
	Web Host	vscodium-reh-web-linux-loong64-1.112.01907.tar.gz

```
cd ~/下载
mkdir -p ~/soft/VSCodeium-1.112
tar -zxvf ~/下载/VSCodeium-linux-loong64-1.112.01907.tar.gz -C ~/soft/VSCodeium-1.112
执行以下命令即可运行 VSCodeium:
cd ~/soft/VSCodeium-1.112
./bin/codium
为方便下次运行，可设置软链接：
sudo cp -r . /usr/local/codium
sudo ln -s /usr/local/codium/bin/codium /usr/local/bin/codium
下次在任何目录下使用下面的指令就可以打开 VSCodeium:
codium
```





3.6 如何安装 Qt

3.6.1 手动安装

目前龙芯平台支持的 QT5 版本为 Qt5.15 Qt6.8

执行以下命令即可安装 Qt5

```
sudo apt install qtcreator qtbase5-dev -y
```

```
ls@ls25-pc:~$ sudo apt install qtcreator qtbase5-dev -y
将要安装:
qtbase5-dev qtcreator

将要安装的依赖:
assistant-qt6 libobjc-14-dev libqt6quickimagegenerator6 qdoc-qt6
clang-19 libpfm4 libqt6test6 qml-qt6
clang-tidy libqt6opengl5-dev libqt6websockets6 qml6-module-qtquick-timeline
clang-tidy-19 libqt6charts6 libxml2-dev qml6-module-qtquick3d
clang-tools-19 libqt6designer6 libyaml-cpp0.8 qml6-module-qtquick3d-particles3d
clangd libqt6designercomponents6 libz3-dev qt6-base-dev-tools
clangd-19 libqt6help6 linguist-qt6 qt6-declarative-dev-tools
designer-qt6 libqt6qmlcompiler6 llvm-19 qt6-documentation-tools
libclang-common-19-dev libqt6quick3d6 llvm-19-dev qt6-l10n-tools
libclang-cpp19 libqt6quick3dassetimport6 llvm-19-linker-tools qt6-qmltooling-plugins
libclang-rt-19-dev libqt6quick3dassetutils6 llvm-19-runtime qt6-tools-dev-tools
libclang1-19 libqt6quick3druntimerender6 llvm-19-tools qtcreator-data
libffi-dev libqt6quick3dutils6 python3-pygments qtcreator-doc
libncurses-dev libqt6quicktest6 python3-yaml

建议安装:
clang-19-doc pkg-config ttf-bitstream-vera libpq-dev clazy valgrind
wasi-libc llvm-19-doc default-libmysqlclient-dev libsqlite3-dev meson python3-pylsp
ncurses-doc python-pygments-doc firebird-dev unixodbc-dev subversion

摘要:
升级: 0, 安装: 57, 卸载: 0, 不升级: 3
下载大小: 157 MB / 159 MB
所需的空间: 953 MB / 7,339 MB 可用

获取:1 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 libqt6help6 loong64 6.8.2-5 [181 kB]
获取:2 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 assistant-qt6 loong64 6.8.2-5 [497 kB]
获取:3 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 libclang-cpp19 loong64 1:19.1.7-3.lnd.1+b1 [12.2 MB]
获取:4 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 libobjc-14-dev loong64 14.2.0-19.lnd.1 [297 kB]
```

执行以下命令即可安装 Qt6

```
sudo apt install qt6-base-dev -y
```

```
ls@ls-pc:~/桌面$ sudo apt install qt6-base-dev
将要安装:
qt6-base-dev

将要安装的依赖:
qmake6 qmake6-bin

摘要:
升级: 0, 安装: 3, 卸载: 0, 不升级: 0
下载大小: 3,639 kB
所需的空间: 34.8 MB / 20.9 GB 可用

是否继续? [Y/n] y
0% [执行中]
获取:1 https://pkg.loongnix.cn/loongnix/25 loongnix-stable/main loong64 qmake6-bin loong64 6.8.2+dfsg-9.lnd.1 [581 kB]
获取:2 https://pkg.loongnix.cn/loongnix/25 loongnix-stable/main loong64 qmake6 loong64 6.8.2+dfsg-9.lnd.1 [150 kB]
获取:3 https://pkg.loongnix.cn/loongnix/25 loongnix-stable/main loong64 qt6-base-dev loong64 6.8.2+dfsg-9.lnd.1 [2,907 kB]
已下载 3,639 kB, 耗时 2秒 (1,754 kB/s)
正在选中未选择的软件包 qmake6-bin。
(正在读取数据库 ... 系统当前共安装有 284753 个文件和目录。)
准备解压 .../qmake6-bin_6.8.2+dfsg-9.lnd.1_loong64.deb ...
```

3.6.2 脚本安装

下载安装脚本 install_Qt.sh 并上传至板卡。



install_Qt.sh

在脚本所在文件目录下执行以下操作：



```
chmod +x install_Qt.sh
./install_Qt.sh
```

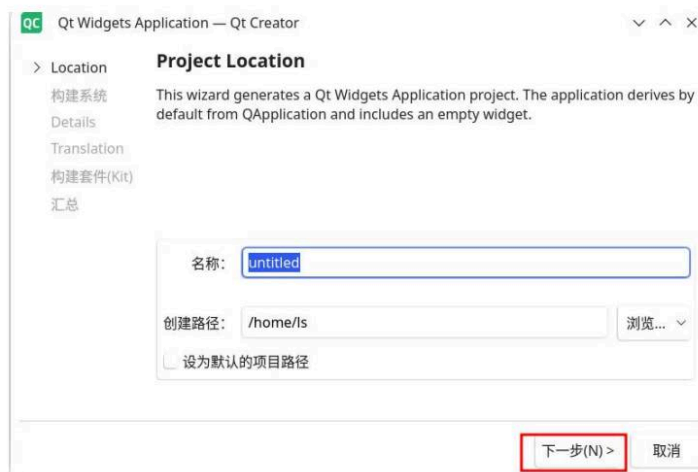
3.6.3 配置 qtcreator

指定 qtcreator qt 版本配置方法：

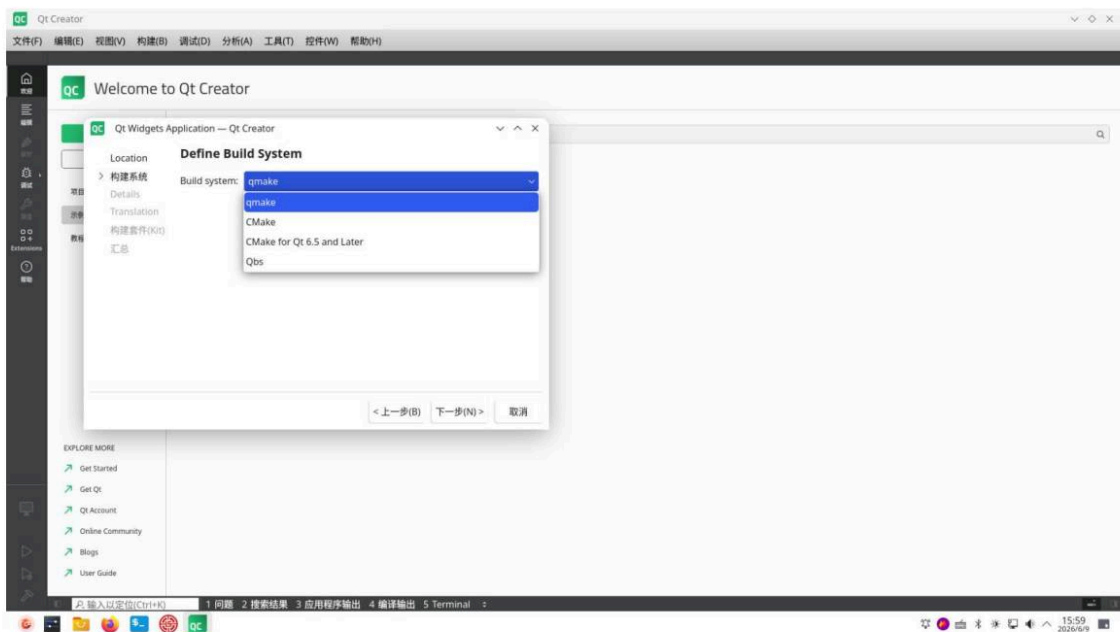
以新建项目配置 qt5.15、qt6.8 为例。打开 qtcreator 后，选择文件->New Project。点击“选择”。



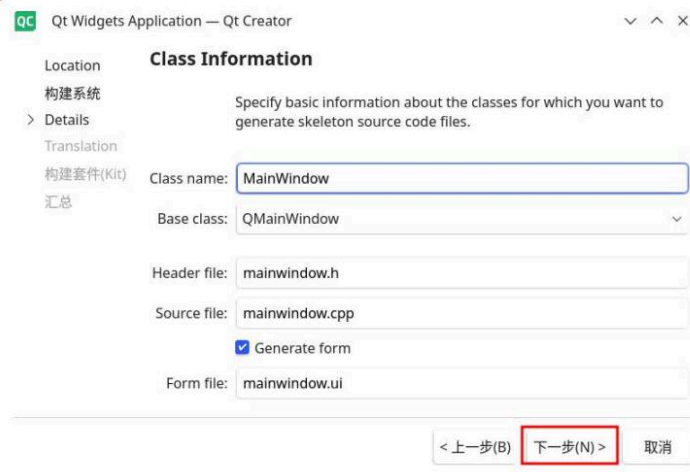
项目名称自定，这里默认不改。点击“下一步”。



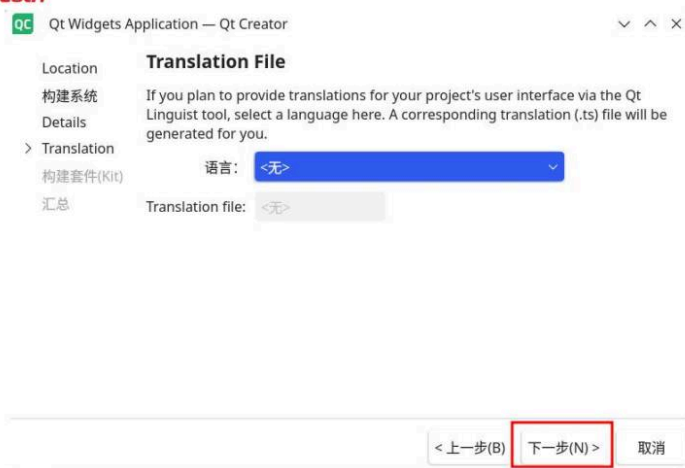
选择 qmake(即为 qt version 5.15.15)。CMake for Qt6.5 and Later (即为 qt6.8 的选择，如果是要用 qt6.8，则选择这一项。)。



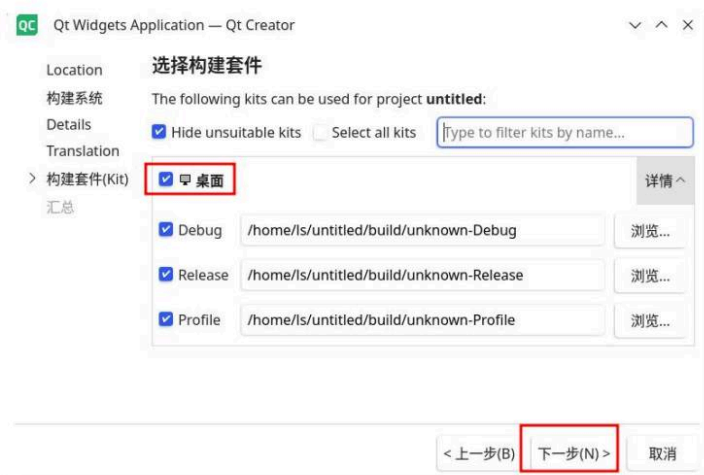
点击“下一步”



点击“下一步”



这里选择“桌面”选项卡，点击“下一步”。



点击“完成”。



如图中绿色三角形位置，点击编译运行，



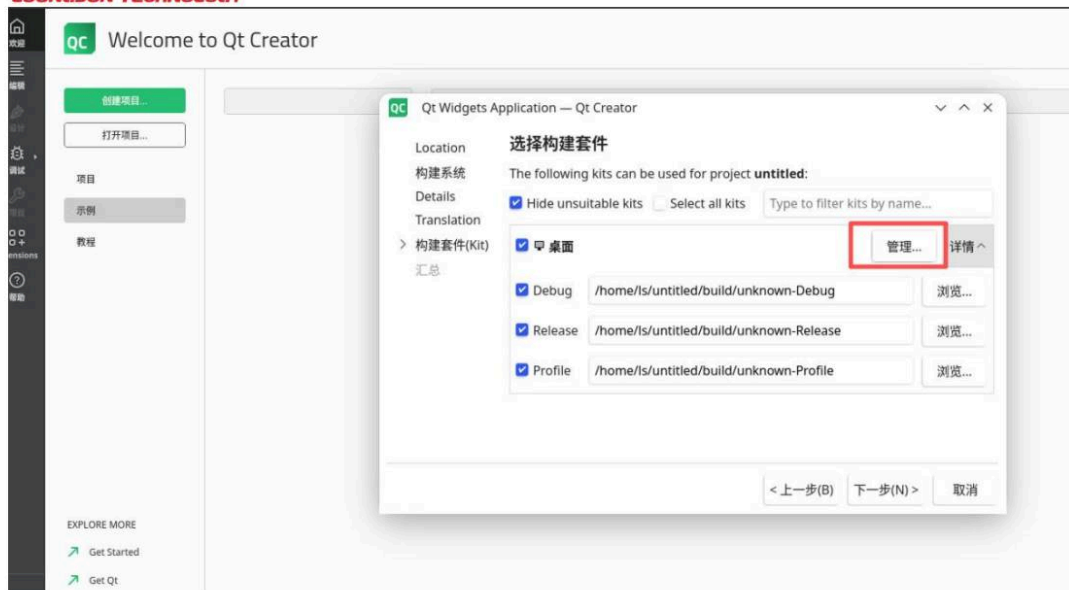
此时可以看到编译输出调用的是 qt5 开发库。

```

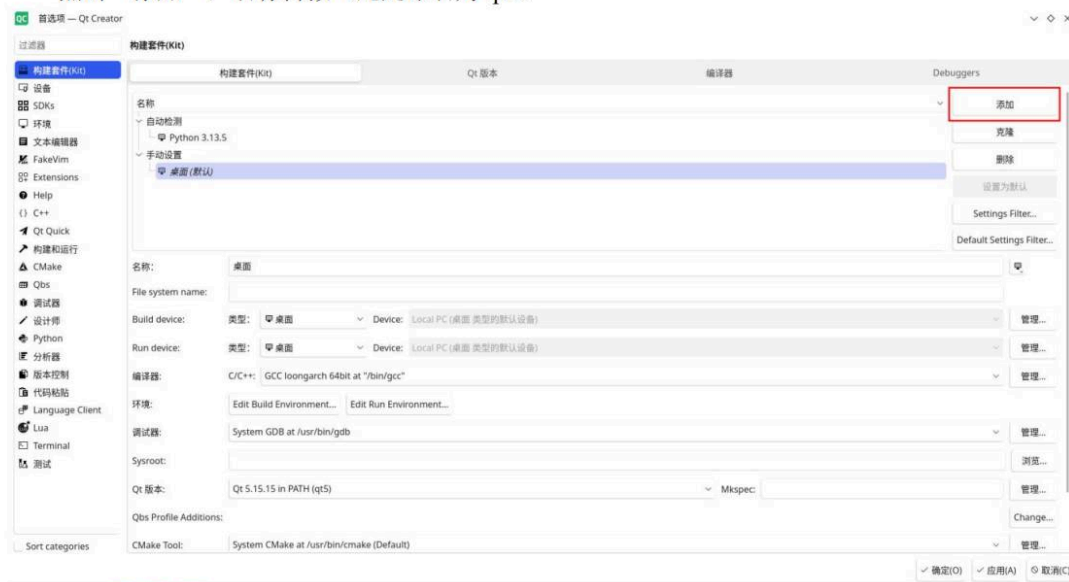
/usr/lib/qt5/bin/uic ../../mainwindow.ui -o ui_mainwindow.h
g++ -c -pipe -g -std=gnu++1z -Wall -Wextra -D_REENTRANT -fPIC -DQT_QML_DEBUG -DQT_WIDGETS_LIB -DQT_GUI_LIB -DQT_CORE_LIB -I../../untitled -I./usr/include/loongarch64-linux-gnu/qt5 -I./usr/include/loongarch64-linux-gnu/qt5/QtWidgets -I./usr/include/loongarch64-linux-gnu/qt5/QtGui -I./usr/include/loongarch64-linux-gnu/qt5/QtCore -I./usr/lib/loongarch64-linux-gnu/qt5/mkspecs/linux-g++ -o main.o ../../main.cpp
g++ -pipe -g -std=gnu++1z -Wall -Wextra -dM -E -o moc_predefs.h /usr/lib/loongarch64-linux-gnu/qt5/mkspecs/features/data/dummy.cpp
g++ -c -pipe -g -std=gnu++1z -Wall -Wextra -D_REENTRANT -fPIC -DQT_QML_DEBUG -DQT_WIDGETS_LIB -DQT_GUI_LIB -DQT_CORE_LIB -I../../untitled -I./usr/include/loongarch64-linux-gnu/qt5 -I./usr/include/loongarch64-linux-gnu/qt5/QtWidgets -I./usr/include/loongarch64-linux-gnu/qt5/QtGui -I./usr/include/loongarch64-linux-gnu/qt5/QtCore -I./usr/lib/loongarch64-linux-gnu/qt5/mkspecs/linux-g++ -o mainwindow.o ../../mainwindow.cpp
/usr/lib/qt5/bin/moc -DQT_QML_DEBUG -DQT_WIDGETS_LIB -DQT_GUI_LIB -DQT_CORE_LIB --include /home/ls/untitled/build/unknown-Debug/moc_predefs.h -I./usr/lib/loongarch64-linux-gnu/qt5/mkspecs/linux-g++ -I/home/ls/untitled -I./usr/include/loongarch64-linux-gnu/qt5 -I./usr/include/loongarch64-linux-gnu/qt5/QtWidgets -I./usr/include/loongarch64-linux-gnu/qt5/QtGui -I./usr/include/loongarch64-linux-gnu/qt5/QtCore -I./usr/include/c++/14 -I./usr/include/loongarch64-linux-gnu/c++/14 -I./usr/include/c++/14/backward -I./usr/lib/gcc/loongarch64-linux-gnu/14/include -I./usr/local/include -I./usr/include/loongarch64-linux-gnu -I./usr/include ../../mainwindow.h -o moc_mainwindow.cpp
g++ -c -pipe -g -std=gnu++1z -Wall -Wextra -D_REENTRANT -fPIC -DQT_QML_DEBUG -DQT_WIDGETS_LIB -DQT_GUI_LIB -DQT_CORE_LIB -I../../untitled -I./usr/include/loongarch64-linux-gnu/qt5 -I./usr/include/loongarch64-linux-gnu/qt5/QtWidgets -I./usr/include/loongarch64-linux-gnu/qt5/QtGui -I./usr/include/loongarch64-linux-gnu/qt5/QtCore -I./usr/lib/loongarch64-linux-gnu/qt5/mkspecs/linux-g++ -o moc_mainwindow.o moc_mainwindow.cpp
g++ -o untitled main.o mainwindow.o moc_mainwindow.o /usr/lib/loongarch64-linux-gnu/libQt5Widgets.so /usr/lib/loongarch64-linux-gnu/libQt5Gui.so /usr/lib/loongarch64-linux-gnu/libQt5Core.so -lGL -lpthread
16:12:56: 进程"/bin/make"正常退出。

```

如果需要使用 qt6.8, 则再新建项目, 其他步骤与前面一样, 在选择构建套件徐选项卡位置点击“管理”,



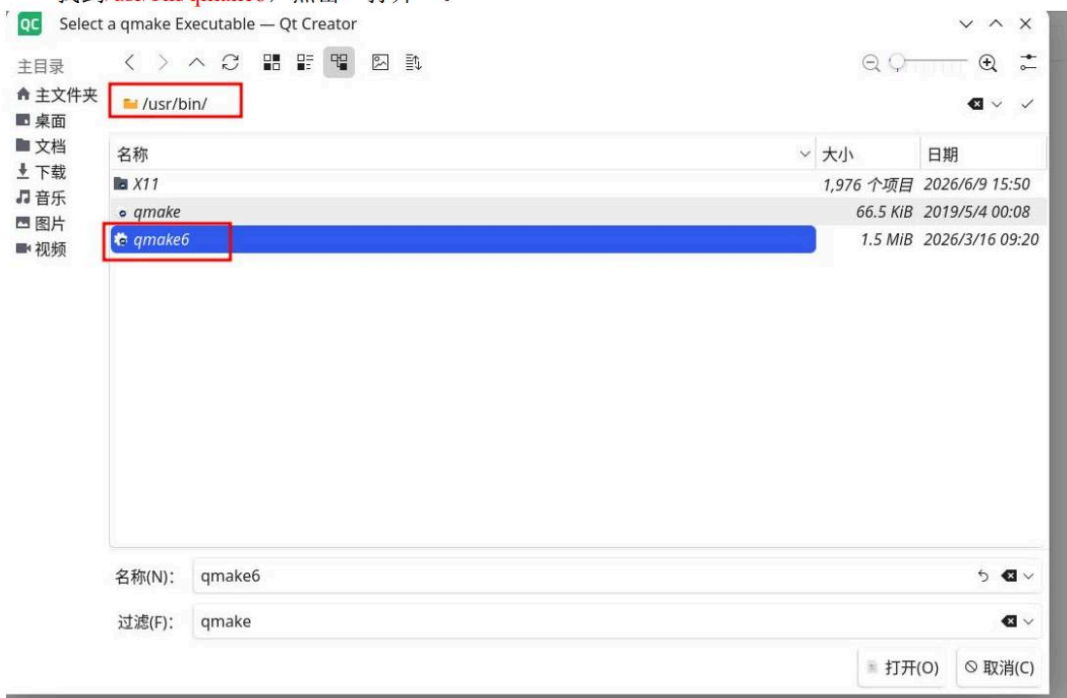
点击“添加”，名称自拟（此处命名为 qt6）



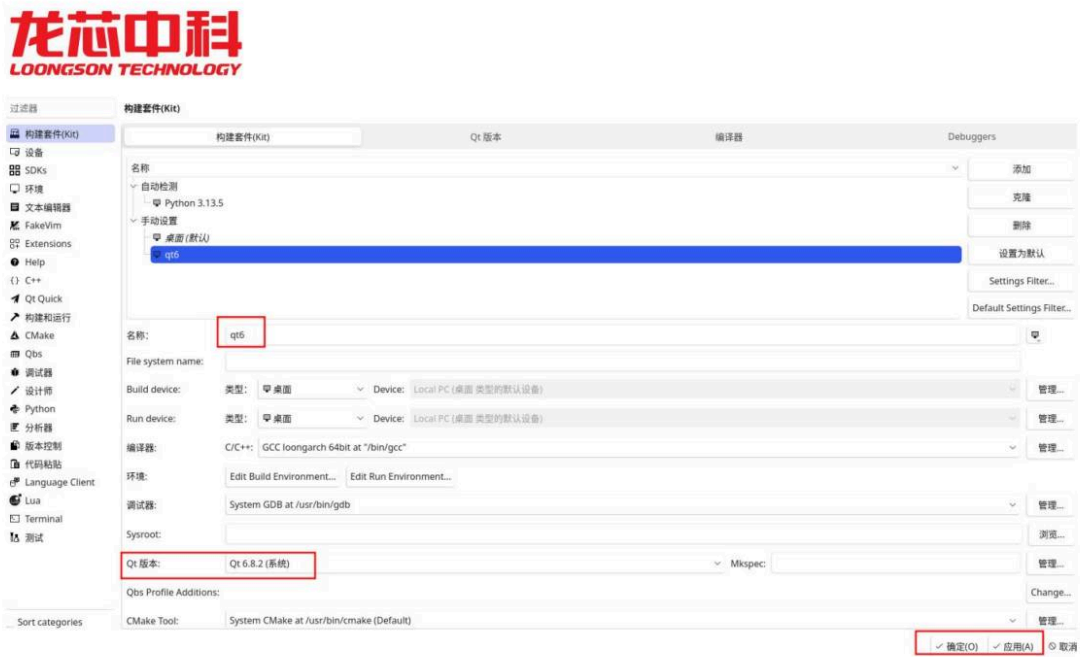
点击“Qt 版本”，点击“添加”。



找到 `/usr/bin/qmake6`，点击“打开”。



接着回到“构建套件（Kit）”选项卡，找到 Qt 版本，指定“Qt 6.8.2(系统)”选项，点击“应用”“确定”。其他步骤与前面 qt5 配置步骤一致。



完成配置后，项目编译信息如下：可以看到 qt6 库成功被调用。



3.7 如何使用龙芯 python 仓库

3.7.1 Python 环境配置及包管理规范

Loongnix 25 集成了 python3.13, 可使用命令查看当前 python 版本信息：

```
python3 --version
```

安装 Python3 的包管理、虚拟环境、编译依赖，让系统能正常开发、运行 Python 项目：

```
sudo apt update
```

```
sudo apt install -y python3-pip python3-venv python3-dev
```

在 Python 3.13 及更高版本中，官方推荐使用虚拟环境进行第三方包的管理，以维护项目依赖隔离和系统环境的稳定性。建议通过以下两种方式安装软件包：

1. 使用 pipx 安装命令行工具

对于可执行的 Python 应用（如 cli 工具），推荐使用 pipx 将其安装至独立虚拟环境，确保依赖



隔离且全局可用：

```
sudo apt install -y pipx
pipx install 包名
```

2.通过 venv 创建项目级虚拟环境

对于项目依赖，应先创建专属虚拟环境，再通过 pip 安装：

```
python -m venv py
source py/bin/activate
pip install 包名
```

之后每次安装 python 包之前都需要先激活虚拟环境（source ~/py/bin/activate）

若需退出虚拟环境，可使用 deactivate 命令：

```
deactivate
```

以上实践符合 Python 打包权威指南（PyPA）规范，能有效避免系统 Python 环境被污染，并确保依赖版本的可控性。

3.7.2 Python 软件包安装方法

Python 软件包的安装可以使用下面三种安装方法：

1. 在线安装方法一（配置永久的下载镜像源）

配置 pip.conf，配置完成后可永久生效。后续安装软件包只需执行“pip3 install 包名”即可，适合板卡可联网、需要频繁安装包且追求长期使用便捷性的场景。

```
mkdir -p ~/.pip
touch ~/.pip/pip.conf
vi ~/.pip/pip.conf
```

添加以下配置内容：

```
[global]
timeout = 60
index-url = https://lpyi.loongnix.cn/loongson/pypi
extra-index-url = https://pypi.tuna.tsinghua.edu.cn/simple
[install]
trusted-host =
    lpyi.loongnix.cn
    pypi.tuna.tsinghua.edu.cn
```

```
ls25@ls25-pc:~$ cat ~/.pip/pip.conf
[global]
timeout = 60
index-url = https://lpyi.loongnix.cn/loongson/pypi
extra-index-url = https://pypi.tuna.tsinghua.edu.cn/simple
[install]
trusted-host =
    lpyi.loongnix.cn
    pypi.tuna.tsinghua.edu.cn
ls25@ls25-pc:~$
```

以安装 minio 包为例，pip3 install minio，等待安装完成即可。



```
ls25@ls25-pc:~$ python3 -m venv py/
ls25@ls25-pc:~$ ls
公共 模板 视频 图片 文档 下载 音乐 桌面 py test
ls25@ls25-pc:~$ source ~/py/
ls25@ls25-pc:~$ source ~/py/bin/activate
ls25@ls25-pc:~$ pip3 install minio
Looking in indexes: https://lpypi.loongnix.cn/loongson/pypi, https://pypi.tuna.tsinghua.edu.cn/simple
Collecting minio
  Using cached https://pypi.tuna.tsinghua.edu.cn/packages/3e/9a/b697530a882588a84db616580f2ba5d1d515c815e11c30d219145afeec87/minio-7.2.2
0-py3-none-any.whl (93 kB)
Collecting argon2-cffi (from minio)
  Using cached https://pypi.tuna.tsinghua.edu.cn/packages/4f/d3/a8b22fa575b297cd6e3e3b0155c7e25db170edf1c74783d6a31a2490b8d9/argon2_cffi
-25.1.0-py3-none-any.whl (14 kB)
Collecting certifi (from minio)
  Using cached https://pypi.tuna.tsinghua.edu.cn/packages/70/7d/9bc192684cea499815ff478dfcdc13835ddf401365057044fb721ec6bddd/certifi-202
5.11.12-py3-none-any.whl (159 kB)
Collecting pycryptodome (from minio)
  Using cached pycryptodome-3.23.0-cp37-abi3-linux_loongarch64.whl
Collecting typing-extensions (from minio)
  Using cached https://pypi.tuna.tsinghua.edu.cn/packages/18/67/36e9267722cc0a4b9f15c7f3441c2363321a3ea07da7ae0c0707beb2a9c/typing_exte
nsions-4.15.0-py3-none-any.whl (44 kB)
Collecting urllib3 (from minio)
  Using cached https://pypi.tuna.tsinghua.edu.cn/packages/a7/c2/fe1e2489ae3122415c51f387e221dd0773709bad6c6cdaa599e8a2c5185/urllib3-2.5
.0-py3-none-any.whl (129 kB)
Collecting argon2-cffi-bindings (from argon2-cffi->minio)
  Using cached argon2_cffi_bindings-25.1.0-cp313-abi3-linux_loongarch64.whl
Collecting cffi>=1.0.1 (from argon2-cffi-bindings->argon2-cffi->minio)
  Using cached http://lpypi.loongnix.cn/loongson/pypi/%2Bf/35c/158eb626e5aca/cffi-2.0.0-cp313-cp313-manylinux_2_38_loongarch64.whl (226
kB)
Collecting pycparser (from cffi>=1.0.1->argon2-cffi-bindings->argon2-cffi->minio)
  Using cached https://pypi.tuna.tsinghua.edu.cn/packages/a0/e3/59cd50310fc9b59512193629e1984cf95e5c8ae6e5d8c69532ccc65a7fe/pycparser-2
.23-py3-none-any.whl (118 kB)
Installing collected packages: urllib3, typing-extensions, pycryptodome, pycparser, certifi, cffi, argon2-cffi-bindings, argon2-cffi, mi
nio
Successfully installed argon2-cffi-25.1.0 argon2-cffi-bindings-25.1.0 certifi-2025.11.12 cffi-2.0.0 minio-7.2.20 pycparser-2.23 pycrypto
dome-3.23.0 typing-extensions-4.15.0 urllib3-2.5.0
```

2. 在线安装方法二（单次指定下载镜像源路径）

只需在命令行中直接携带镜像参数即可完成安装，不过配置仅单次生效。操作上每次安装都需要输入完整参数（仅需替换包名），适合板卡可联网、偶尔安装包，且不愿修改系统配置文件的使用场景。

以安装 pandas 包为例（安装其他包，只需要把 pandas 换成相应名称即可）

```
pip3 install pandas -i https://lpypi.loongnix.cn/loongson/pypi --trusted-host
lpypi.loongnix.cn --extra-index-url https://pypi.org/simple --trusted-host pypi.org
```

```
ls25@ls25-pc:~$ cd py/
ls25@ls25-pc:~/py$ source ~/py/bin/activate
(py) ls25@ls25-pc:~/py$ pip3 install pandas -i https://lpypi.loongnix.cn/loongson/pypi --trusted-host lpypi.loongnix.cn --extra-index-u
rl https://pypi.org/simple --trusted-host pypi.org
Looking in indexes: https://lpypi.loongnix.cn/loongson/pypi, https://pypi.tuna.tsinghua.edu.cn/simple, https://pypi.org/simple
Collecting pandas
  Using cached http://lpypi.loongnix.cn/loongson/pypi/%2Bf/038/32a37b2a6cc45/pandas-2.3.3-cp313-cp313-manylinux_2_36_loongarch64.manylin
ux_2_38_loongarch64.whl (12.2 MB)
Collecting numpy>=1.26.0 (from pandas)
  Downloading numpy-2.3.5.tar.gz (20.6 MB)
    Installing build dependencies ... done
    Getting requirements to build wheel ... done
    Installing backend dependencies ... done
    Preparing metadata (pyproject.toml) ... done
Collecting python-dateutil>=2.8.2 (from pandas)
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting pytz>=2020.1 (from pandas)
  Downloading pytz-2025.2-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas)
  Downloading tzdata-2025.2-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas)
  Downloading six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Downloaded python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloaded six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloaded tzdata-2025.2-py2.py3-none-any.whl (347 kB)
Building wheels for collected packages: numpy
  Building wheel for numpy (pyproject.toml) ... done
  Created wheel for numpy: filename=numpy-2.3.5-cp313-cp313-linux_loongarch64.whl size=8547288 sha256=91e1d69aa7cc354f198dd6fd821cfe3724
007e67aa5f482ff7eff7c75d7e1083
  Stored in directory: /home/ls25/.cache/pip/wheels/e1/94/10/27deea10acc6fcdce1e8500bbf65510dd06f0b2409e10f104
Successfully built numpy
Installing collected packages: pytz, tzdata, six, numpy, python-dateutil, pandas
Successfully installed numpy-2.3.5 pandas-2.3.3 python-dateutil-2.9.0.post0 pytz-2025.2 six-1.17.0 tzdata-2025.2
```



opencv-python-4.10.0.84	opencv_python-4.10.0.84-cp310-cp310-linux_loongarch64.whl
opencv-python-headless-4.10.0.84	opencv_python-headless-4.10.0.84-cp39-cp39-linux_loongarch64.whl
openpyxl-3.0.10	openpyxl-3.0.10-py2.py3-none-any.whl
opt-einsum-3.3.0	opt_einsum-3.3.0-py3-none-any.whl
orjson-3.10.18	orjson-3.10.18-cp310-cp310-manylinux_2_28_loongarch64.whl
ormsgpack-1.5.0	ormsgpack-1.5.0-cp310-cp310-manylinux_2_28_loongarch64.whl
packaging-21.3	packaging-21.3-py3-none-any.whl
paddlepaddle-2.6.1	paddlepaddle-2.6.1-cp38-cp38-linux_loongarch64.whl
pandas-2.2.3	pandas-2.2.3-cp310-cp310-manylinux_2_28_loongarch64.whl
pandocfilters-1.4.3	pandocfilters-1.4.3-py3-none-any.whl
paramiko-2.11.0	paramiko-2.11.0-py2.py3-none-any.whl
parso-0.8.2	parso-0.8.2-py2.py3-none-any.whl
patchelf-0.17.2.2	patchelf-0.17.2.2-py3-none-manylinux_2_28_loongarch64.whl
pathspec-0.8.1	pathspec-0.8.1-py2.py3-none-any.whl
patool-1.12	patool-1.12-py2.py3-none-any.whl
pdf2image-1.16.3	pdf2image-1.16.3-py3-none-any.whl
pdfminer-six-20221105	pdfminer-six-20221105-py3-none-any.whl
peewee-3.15.0	peewee-3.15.0-py3-none-any.whl
perin-parser-0.0.12	perin_parser-0.0.12-py3-none-any.whl
pexpect-4.8.0	pexpect-4.8.0-py2.py3-none-any.whl
phrasetree-0.0.8	phrasetree-0.0.8-py3-none-any.whl
pickleshare-0.7.5	pickleshare-0.7.5-py2.py3-none-any.whl

3. 离线安装（.whl 文件）

提前下载适配 LoongArch64 架构的二进制包并在本地完成安装，完全无网络依赖。需要手动下载包并上传至板卡，且包的依赖关系也需手动处理，主要适用于板卡无网络、处于内网环境或不适用于联网的场景。

访问龙芯镜像源：<https://lpypi.loongnix.cn/loongson/pypi>，下载相关的.whl 文件。

以安装 aiohttp-3.13.2 为例，先下载相应的.whl 文件（注意选用 python 对应版本的 loongarch64 包）

The screenshot shows the Loongson PyPI index page. The 'Releases' section lists various packages for different architectures. The package 'aiohttp-3.13.2-cp313-cp313-manylinux_2_36_loongarch64.manylinux_2_38_loongarch64.whl' is highlighted in red, indicating it is the correct package for LoongArch64 architecture.

下载完成后，在命令行输入（cp313 指代 python3.13）

```
pip3
install ./aiohttp-3.13.2-cp313-cp313-manylinux_2_36_loongarch64.manylinux_2_38_loongarch64.whl
```



```
ls25@ls25-pc:~/下载$ source ~/py/bin/activate
(py) ls25@ls25-pc:~/下载$ pip3 install ./aiohttp-3.13.2-cp313-cp313-manylinux_2_36_loongarch64.manylinux_2_38_loongarch64.whl
aiohttp-3.13.2-cp313-cp313-manylinux_2_36_loongarch64.manylinux_2_38_loongarch64.whl
code-oss-1.100.3-1751441829_loongarch64.deb
codium-1.100.33714_loongarch64.deb
(py) ls25@ls25-pc:~/下载$ pip3 install ./aiohttp-3.13.2-cp313-cp313-manylinux_2_36_loongarch64.manylinux_2_38_loongarch64.whl
Looking in indexes: https://pypi.loongnix.cn/loongson/pypi, https://pypi.tuna.tsinghua.edu.cn/simple
Processing ./aiohttp-3.13.2-cp313-cp313-manylinux_2_36_loongarch64.manylinux_2_38_loongarch64.whl
Collecting aiohappyeyeballs>=2.5.0 (from aiohttp==3.13.2)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/0f/15/5bf3b99495fb160b63f95972b81750f18f7f4e02ad051373b669d17d44f2/aiohappyeyeballs-2.6.1-py3-none-any.whl (15 kB)
Collecting aiosignal>=1.4.0 (from aiohttp==3.13.2)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/fb/76/641ae371508676492379f16e2fa48f4e2c11741bd63c48be4b12a6b09c9ba/aiosignal-1.4.0-py3-none-any.whl (7.5 kB)
Collecting attrs>=17.3.0 (from aiohttp==3.13.2)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/3a/2a/7cc015f5b9f5db42b7d48157e23356022889fc354a2813c15934b7cb5c0e/attrs-25.4.0-py3-none-any.whl (67 kB)
Collecting frozenlist>=1.1.1 (from aiohttp==3.13.2)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/9a/9a/e35b4a917281c0b8419d4207f4334c8e8c5dbf4f3f5f9ada73958d937dccc/frozenlist-1.8.0-py3-none-any.whl (13 kB)
Collecting multidict>=7.0.0 (from aiohttp==3.13.2)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/b7/da/7d22601b625e241d4f23ef1ebff8acfc60da633c9e7e7922e24d10f592b3/multidict-6.7.0-py3-none-any.whl (12 kB)
Collecting propcache>=0.2.0 (from aiohttp==3.13.2)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/5b/5a/bc7b4a4ef808fa59a816c17b20c4bef6884daebddf627f2a161da67da19/propcache-0.4.1-py3-none-any.whl (13 kB)
Collecting yarl<2.0, >=1.17.0 (from aiohttp==3.13.2)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/73/ae/b48f95715333080afb75a4504487cbe142cae1268afc482d06692d605ae6/yarl-1.22.0-py3-none-any.whl (46 kB)
Collecting idna>=2.0 (from yarl<2.0, >=1.17.0->aiohttp==3.13.2)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/0e/61/66938bb5f5c52dbdf84594873d5b51fb1f7c7794e9c0f5bd885f30bc507b/idna-3.11-py3-none-any.whl (71 kB)
Installing collected packages: propcache, multidict, idna, frozenlist, attrs, aiohappyeyeballs, yarl, aiosignal, aiohttp
Successfully installed aiohappyeyeballs-2.6.1 aiohttp-3.13.2 aiosignal-1.4.0 attrs-25.4.0 frozenlist-1.8.0 idna-3.11 multidict-6.7.0 propcache-0.4.1 yarl-1.22.0
(py) ls25@ls25-pc:~/下载$
```

3.7.3 使用 pyenv 安装和管理多版本 Python

Pyenv 是轻量的 Python 多版本管理工具，可以在不污染系统环境的前提下，安装、切换任意 Python 版本，支持全局生效和项目单独生效，满足各种的开发需求。

1. 安装 pyenv

执行以下命令，通过 Git 克隆 pyenv 源码到用户目录安装 pyenv：

```
git clone https://github.com/pyenv/pyenv.git ~/.pyenv
```

```
ls@ls-pc:~$ git clone https://github.com/pyenv/pyenv.git ~/.pyenv
正克隆到 '/home/ls/.pyenv' ...
remote: Enumerating objects: 27380, done.
remote: Counting objects: 100% (10/10), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 27380 (delta 7), reused 5 (delta 5), pack-reused 27370 (from 3)
接收对象中: 100% (27380/27380), 6.27 MiB | 3.76 MiB/s, 完成。
处理 delta 中: 100% (18399/18399), 完成。
```

将 pyenv 配置写入系统环境变量并让其马上生效，此时终端能永久识别 pyenv 命令：

```
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.bashrc
echo 'command -v pyenv >/dev/null || export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.bashrc
echo 'eval "$(pyenv init -)"' >> ~/.bashrc
source ~/.bashrc
```

后续可使用 pyenv 安装不同的 Python 版本，以满足项目开发需求。

2. 使用 pyenv 安装 Python

可先执行命令查看可安装的 Python 版本：

```
pyenv install --list
```

然后执行 pyenv install 命令进行安装。

下面以安装 Python 3.12.1 和 Python 3.11.1 为例，命令如下：

```
pyenv install 3.12.1
pyenv install 3.11.1
```



安装完成后，可使用命令 `pyenv versions` 查看已安装的版本。

```
ls@lab1:~$ pyenv install 3.12.1
Downloading Python-3.12.1.tar.xz ...
→ https://www.python.org/ftp/python/3.12.1/Python-3.12.1.tar.xz
Installing Python-3.12.1 ...
Installed Python-3.12.1 to /home/ls/.pyenv/versions/3.12.1
ls@lab1:~$ pyenv install 3.11.1
Downloading Python-3.11.1.tar.xz ...
→ https://www.python.org/ftp/python/3.11.1/Python-3.11.1.tar.xz
Installing Python-3.11.1 ...
patching file setup.py
Hunk #1 succeeded at 306 (offset -1 lines).
Installed Python-3.11.1 to /home/ls/.pyenv/versions/3.11.1
ls@lab1:~$ pyenv versions
* system (set by /home/ls/.pyenv/version)
 3.11.1
 3.12.1
```

3. 不同 Python 版本间的切换

可根据项目开发需要切换不同的 Python 版本，pyenv 支持全局切换和当前目录单独切换。

如需在所有终端都使用刚安装的 Python 3.12.1 版本，可以使用全局切换命令 `pyenv global`

`pyenv global 3.12.1`

若想切换回系统自带的 Python 版本，则可以使用命令：

`pyenv global system`

```
ls@lab1:~$ pyenv global 3.12.1
ls@lab1:~$ python3 --version
Python 3.12.1
ls@lab1:~$ pyenv global system
ls@lab1:~$ python3 --version
Python 3.13.5
```

若只需在当前的项目开发目录下使用刚安装的 Python 3.12.1 版本，则可先创建和进入项目目录使用 `pyenv local` 命令切换 python 版本：

`mkdir -p ~/文档/myproject`

`cd ~/文档/myproject`

`pyenv local 3.12.1`

```
ls@lab1:~$ mkdir -p ~/文档/myproject
ls@lab1:~$ cd ~/文档/myproject
ls@lab1:~/文档/myproject$ pyenv local 3.12.1
ls@lab1:~/文档/myproject$ python3 --version
Python 3.12.1
```

此时，只有所在项目使用新的 Python 版本，系统其余地方默认使用当前系统的 python 版本。

按需选择 python 版本后，后续开发可按照 3.7.1 节和 3.7.2 节介绍的步骤，创建和激活虚拟环境，然后再进行相关软件包的安装。

3.7.4 脚本安装

下载安装脚本 `install_python_env.sh` 并上传至板卡。



`install_python_env.sh`

在脚本所在文件目录下执行以下操作：

`chmod +x install_python_env.sh`

`./install_python_env.sh`



3.8 如何安装 docker 和使用龙芯 docker 仓库

3.8.1 安装 docker

```
sudo apt install docker.io -y
```

```
(py) ls25@ls25-pc:~/下载$ sudo apt install docker.io -y
将要安装:
docker.io

将要安装的依赖:
containerd docker-buildx docker-cli iptables libip4tc2 libip6tc2 runc tini

建议安装:
containernetworking-plugins aufs-tools debootstrap rootlesskit zfs-fuse firewallld
docker-doc cgroupfs-mount rinse xfsprogs | zfsutils-linux

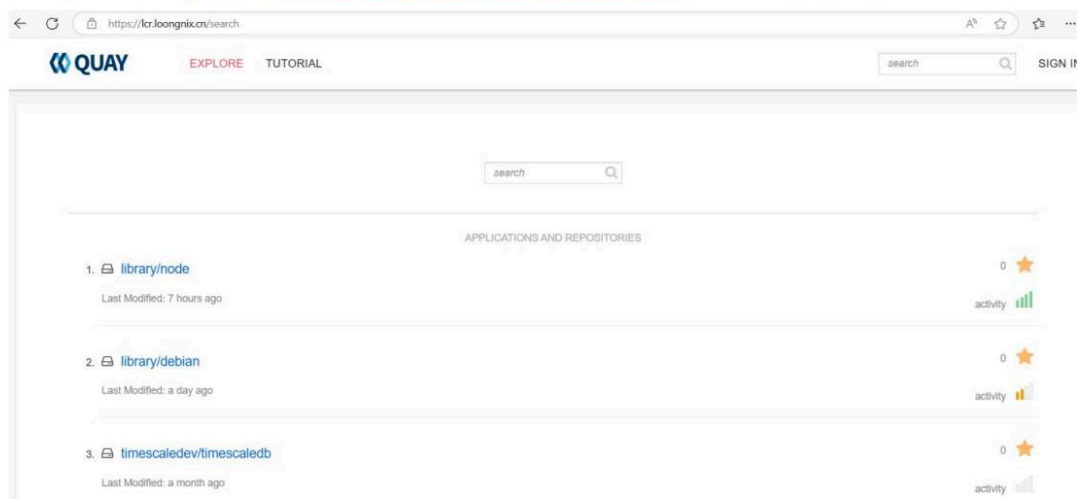
推荐安装:
needrestart criu

摘要:
升级: 0, 安装: 9, 卸载: 0, 不升级: 3
下载大小: 56.8 MB
所需的空: 272 MB / 6,357 MB 可用

获取:1 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 runc loong64 1.1.15+ds1-2 [2,829 kB]
获取:2 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 containerd loong64 1.7.24~ds1-6 [20.1 MB]
获取:3 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 libip4tc2 loong64 1.8.11-2 [20.4 kB]
获取:4 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 libip6tc2 loong64 1.8.11-2 [20.7 kB]
获取:5 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 iptables loong64 1.8.11-2 [364 kB]
获取:6 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 tini loong64 0.19.0-3 [254 kB]
获取:7 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 docker.io loong64 26.1.5+dfsg1-9 [15.3 MB]
获取:8 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 docker-buildx loong64 0.13.1+ds1-3 [11.5 MB]
获取:9 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 docker-cli loong64 26.1.5+dfsg1-9 [6,425 kB]
已下载 56.8 MB, 耗时 41秒 (5,049 kB/s)
正在选中未选择的软件包 runc
(正在读取数据库 ... 系统当前共安装有 262006 个文件和目录。)
准备解压 .../0-runc_1.1.15+ds1-2_loong64.deb ...
正在解压 runc (1.1.15+ds1-2) ...
```

3.8.2 使用龙芯 docker 仓库

龙芯容器镜像仓库地址: <https://lcr.loongnix.cn/search> 根据需要拉取相应镜像。



docker images 查看 docker 镜像

docker ps 显示运行中的容器



拉取镜像(以 `alpine:3.21` 为例): `docker pull lcr.loongnix.cn/library/alpine:3.21`

启动容器: `docker run -d -it --name al lcr.loongnix.cn/library alpine:3.21`

```
root@ls25-pc: /home/ls25# docker images
REPOSITORY      TAG                IMAGE ID          CREATED          SIZE
root@ls25-pc: /home/ls25# docker ps
CONTAINER ID      IMAGE               COMMAND           CREATED          STATUS          PORTS           NAMES
root@ls25-pc: /home/ls25# docker pull lcr.loongnix.cn/library/alpine:3.21
3.21: Pulling from library/alpine
9f478f2ace4d: Pull complete
Digest: sha256:b4a698dc34dbd5079eea01b7ea3488452a9a1c3bc8aad11033ea2cc978f49ffc
Status: Downloaded newer image for lcr.loongnix.cn/library/alpine:3.21
lcr.loongnix.cn/library/alpine:3.21
```

在指定的 Docker 容器内启动一个交互式的 bash shell: `docker exec -it edb5df8ce7b8 /bin/sh`

```

root@ls25-pc:/home/ls25# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS        NAMES
edb5df8ce7b8   lcr_loongnix.cn/library/alpine:3.21  "/bin/sh"              5 minutes ago  Up 5 minutes  al

root@ls25-pc:/home/ls25# docker exec -it edb5df8ce7b8 /bin/sh

/ # apk add fastfetch
fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/main/loongarch64/APKINDEX.tar.gz
fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/community/loongarch64/APKINDEX.tar.gz
(1/2) Installing hwdata-pci (0.393-r0)
(2/2) Installing fastfetch (2.36.1-r0)
Executing busybox-1.37.0-r8.trigger
OK: 12 MiB in 17 packages

/ # fastfetch

      .hddddd
:ddddd
/ddddd
+ddddd
'sddddd
yddddd++hddddd
.hddddd++   +dddh:-sddddd
hddddd++   +y:  .sddddd
ddddd++   //:-sddddd
ddddd++   /hddh/:-sddddd
dddh+   /+/ddddd/:-sddddd
ddd+   /o:ddddd/:-oy-.ydd
hdddyo+ohddyosddddd+oydddy+ohddh
.hddddd
yddddd
sddddd
+ddddd
/ddddd
:ddddd
.hddddd

root@edb5df8ce7b8
-----
05: Alpine Linux v3.21 loongarch64
Host: Loongson-2K3000-EVB
Kernel: Linux 6.5.52-loong64
Uptime: 58 mins
Packages: 17 (apk)
Shell: sh
Display (TF2411): 1920x1080 @ 60 Hz in 24" [External]
Terminal: xterm
Terminal Font: fixed (8.0pt)
CPU: Loongson-2K3000 (8) @ 2.50 GHz
GPU: Loongson Technology LLC Device 7A46 (VGA compatible)
Memory: 5.41 GiB / 15.50 GiB (35%)
Swap: Disabled
Disk (/): 21.28 GiB / 28.39 GiB (75%) - overlay
Local IP (eth0): 172.17.0.2/16
Locale: C

```

安装 fastfetch 软件包 `apk add fastfetch`

安装完成后输入 fastfetch 运行软件，至此 docker 可正常使用。

3.8.3 制作镜像

介绍基于 Loongnix 系统镜像 loongnix/loongnix-server 通过 Dockerfile 构建镜像得方法。下面以构建 MariaDB 镜像为例说明操作步骤。

1、创建 Docker 配置文件，解除 Docker 安全限制

```
sudo tee /etc/docker/daemon.json << EOF
{
  "seccomp-profile": "unconfined"
}
EOF
```

重启 Docker 让配置生效

```
sudo systemctl daemon-reload
sudo systemctl restart docker
```

2、创建目录



```
mkdir -p ~/mysql-loongnix
```

```
cd ~/mysql-loongnix
```

3、生成 Dockerfile（以下为一条指令）

```
cat > Dockerfile << EOF
```

```
FROM lcr.loongnix.cn/loongnix/loongnix-server:23.1
```

```
# 安装
```

```
RUN yum install -y mariadb-server && yum clean all
```

```
# 初始化
```

```
RUN mariadb-install-db --user=mysql --datadir=/var/lib/mysql || true
```

```
# 强制权限
```

```
RUN chown -R mysql:mysql /var/lib/mysql || true
```

```
# 启动命令
```

```
CMD ["/usr/libexec/mysqld", "--user=mysql", "--console"]
```

```
EOF
```

4、构建镜像

```
sudo docker build -t loongnix-mysql .
```

5、启动容器

```
sudo docker run -d --name loongnix-mysql loongnix-mysql
```

6、查看是否运行成功

```
sudo docker ps
```

```
ls@lab1:~/mysql-loongnix$ sudo docker build -t loongnix-mysql .
[+] Building 70.1s (8/8) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 405B                               0.0s
=> [internal] load metadata for lcr.loongnix.cn/loongnix/loongnix-server:23.1 0.7s
=> [internal] load .dockerignore                                  0.1s
=> => transferring context: 2B                                       0.0s
=> [1/4] FROM lcr.loongnix.cn/loongnix/loongnix-server:23.1@sha256:2bc72f390731a3fea91224b569fb80b096c0f86 43.6s
=> => resolve lcr.loongnix.cn/loongnix/loongnix-server:23.1@sha256:2bc72f390731a3fea91224b569fb80b096c0f866 0.0s
=> => sha256:2bc72f390731a3fea91224b569fb80b096c0f8664857e27ddf59bfc2797d68c7 530B / 530B 0.0s
=> => sha256:70f1cda563aa29b0b2d32d7556cfe9f777f8309f23f9c9c967222135901ba900 1.76kB / 1.76kB 0.0s
=> => sha256:9f1ec78e664d16ac123c2ece55a56f3fb43e2e66394bfe69b4d5cc6144af4ea9 109.03MB / 109.03MB 33.3s
=> => extracting sha256:9f1ec78e664d16ac123c2ece55a56f3fb43e2e66394bfe69b4d5cc6144af4ea9 9.8s
=> [2/4] RUN yum install -y mariadb-server && yum clean all 18.2s
=> [3/4] RUN mariadb-install-db --user=mysql --datadir=/var/lib/mysql || true 4.0s
=> [4/4] RUN chown -R mysql:mysql /var/lib/mysql || true 1.5s
=> exporting to image 1.6s
=> => exporting layers 1.6s
=> => writing image sha256:12b15bcd80fc2600ed7d814040ce49aff9b3e0288edf772029101f0ad6dd5b77 0.0s
=> => naming to docker.io/library/loongnix-mysql 0.0s
ls@lab1:~/mysql-loongnix$ sudo docker run -d --name loongnix-mysql loongnix-mysql
a698e698d36c9b6cce0d05ac63c62e1058f5d59b4097b2a2d113bf9961d7e63c
ls@lab1:~/mysql-loongnix$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a698e698d36c	loongnix-mysql	"/usr/libexec/mysqld..."	18 seconds ago	Up 16 seconds		loongnix-mysql

测试:

```
sudo docker exec -it loongnix-mysql bash
```



```
ls@lab1:~$ sudo docker exec -it loongnix-mysql bash
[root@a698e698d36c /]# mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 4
Server version: 10.6.13-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| test |
+-----+
5 rows in set (0.004 sec)
```

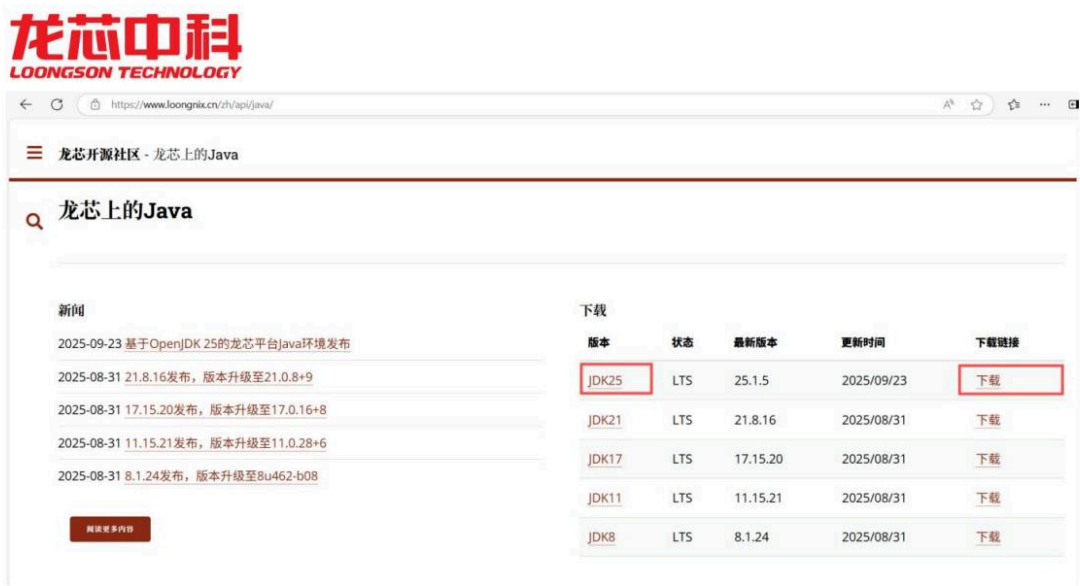
3.9 如何安装 java

3.9.1 手动安装

先安装 java-common，输入命令：`sudo apt-get install java-common`

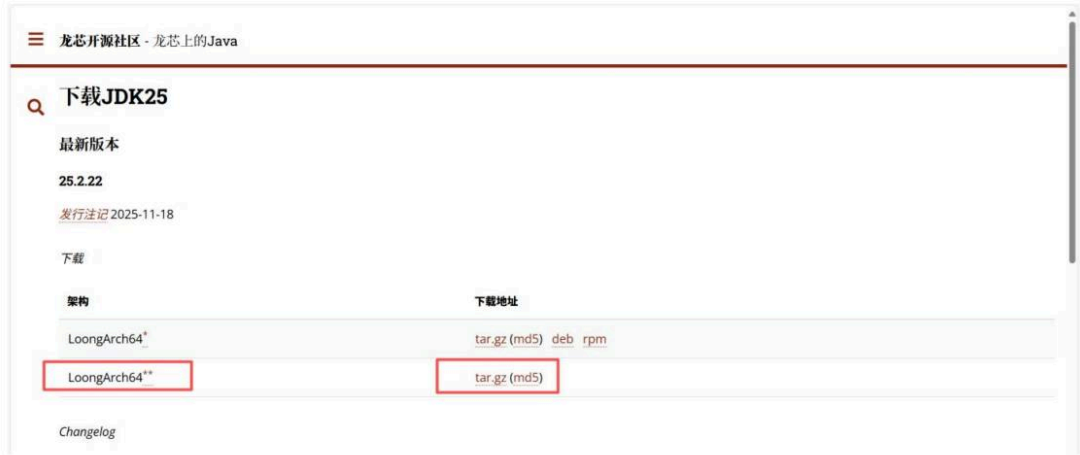
```
loongsong@loongson-pc:~/下载$ sudo apt-get install java-common
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
下列软件包是自动安装的并且现在不需要了:
  cracklib-runtime libboost-python1.67.0 libcrack2 libkpmcore7 libpwquality-common libpwquality1 squashfs-tools
使用'sudo apt autoremove'来卸载它(它们)。
建议安装:
  default-jre
下列【新】软件包将被安装:
  java-common
升级了 0 个软件包，新安装了 1 个软件包，要卸载 0 个软件包，有 0 个软件包未被升级。
有 1 个软件包没有被完全安装或卸载。
需要下载 14.6 kB 的归档。
解压后会消耗 36.9 kB 的额外空间。
获取:1 https://pkg.loongnix.cn/loongnix/20 DaoXiangHu-stable/main loongarch64 java-common all 0.71.lnd.4 [14.6 kB]
已下载 14.6 kB，耗时 1秒 (9,830 B/s)
正在选中未选择的软件包 java-common。
(正在读取数据库 ... 系统当前共安装有 194046 个文件和目录。)
准备解压 .../java-common_0.71.lnd.4_all.deb ...
正在解压 java-common (0.71.lnd.4) ...
正在设置 java-common (0.71.lnd.4) ...
```

以下载 JDK25 为例，<https://www.loongnix.cn/zh/api/java/>



选中 tar 包，这里以 tar 包安装为例（参考安装说明：<https://docs.loongnix.cn/java/install.html>）

如果点击 JDK25，就选中 LoongArch64**：tar 包，如下图所示：



如果点击 下载，就选中 LoongArch64[2]：tar 包，如下图所示：



获取龙芯平台JDK25

当前版本25.2.22

请选择要下载的安装包：

LoongArch64^[1]:

☐ TAR ☐ DEB ☐ RPM

LoongArch64^[2]:

☒ TAR

下载

下载的 tar 包默认在 `~/下载` 目录下。

```
cd ~/下载
sudo tar xzf loongson25.2.22-fx-jdk25.0.1_8-linux-loongarch64-glibc2.34.tar.gz -C /opt/
```

```
ls25@ls25-pc:~/下载$ sudo tar xzf loongson25.2.22-fx-jdk25.0.1_8-linux-loongarch64-glibc2.34.tar.gz -C /opt/
[sudo] ls25 的密码:
```

之后 `cd /opt/jdk-25.0.1` 执行 `bin/java -version` 查看版本信息，可以看到 java 安装成功。

```
ls25@ls25-pc:/opt/jdk-25.0.1$ bin/java --version
openjdk 25.0.1 2025-10-21
OpenJDK Runtime Environment Loongson25.2.22 (build 25.0.1+8)
OpenJDK 64-Bit Server VM Loongson25.2.22 (build 25.0.1+8, mixed mode, sharing)
ls25@ls25-pc:/opt/jdk-25.0.1$ pwd
/opt/jdk-25.0.1
ls25@ls25-pc:/opt/jdk-25.0.1$ export PATH=/opt/jdk-25.0.1/bin:$PATH
ls25@ls25-pc:/opt/jdk-25.0.1$ java --version
openjdk 25.0.1 2025-10-21
OpenJDK Runtime Environment Loongson25.2.22 (build 25.0.1+8)
OpenJDK 64-Bit Server VM Loongson25.2.22 (build 25.0.1+8, mixed mode, sharing)
```

如果每次都需要重新声明环境变量 `export PATH`，会非常麻烦，通过将该命令（`export PATH=/opt/jdk-25.0.1/bin:$PATH`）追加到 `~/.bashrc` 文件末尾，后面打开新终端就不需要再声明该环境变量了。

3.9.2 脚本安装

下载安装脚本 `install_java.sh` 并上传至板卡。



在脚本所在文件目录下执行以下操作：

```
chmod +x install_java.sh
./install_java.sh
```



3.10 如何安装 go

3.10.1 手动安装

```
ls25@ls25-pc:~/下载$ sudo apt install golang-go
将要安装:
golang-go
将要安装的依赖:
golang-1.24-go golang-1.24-src golang-src libpkgconf3 pkgconf pkgconf-bin
建议安装:
bzip | brz mercurial subversion
摘要:
升级: 0, 安装: 7, 卸载: 0, 不升级: 3
下载大小: 46.8 MB
所需的空: 258 MB / 5,137 MB 可用
是否继续? [Y/n] y
获取:1 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 golang-1.24-src all 1.24.4-1.lnd.1 [21.4 MB]
获取:2 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 golang-1.24-go loong64 1.24.4-1.lnd.1+b1 [25.3 MB]
获取:3 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 golang-src all 2:1.24-2 [5,136 B]
获取:4 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 golang-go loong64 2:1.24-2 [44.3 kB]
获取:5 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 libpkgconf3 loong64 1.8.1-4 [36.9 kB]
获取:6 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 pkgconf-bin loong64 1.8.1-4 [29.9 kB]
获取:7 http://buildd.loongnix.cn:8000/controlled loongnix-stable/main loong64 pkgconf loong64 1.8.1-4 [26.2 kB]
已下载 46.8 MB, 耗时 10秒 (4,855 kB/s)
正在选中未选择的软件包 golang-1.24-src。
(正在读取数据库 ... 系统当前共安装有 262566 个文件和目录。)
准备解压 .../0-golang-1.24-src 1.24.4-1.lnd.1_all.deb ...
正在解压 golang-1.24-src (1.24.4-1.lnd.1) ...
正在选中未选择的软件包 golang-1.24-go。
准备解压 .../1-golang-1.24-go 1.24.4-1.lnd.1+b1_loong64.deb ...
正在解压 golang-1.24-go (1.24.4-1.lnd.1+b1) ...
正在选中未选择的软件包 golang-src。
准备解压 .../2-golang-src 2%3a1.24~2_all.deb ...
正在解压 golang-src (2:1.24-2) ...
正在选中未选择的软件包 golang-go:loong64。
准备解压 .../3-golang-go 2%3a1.24~2_loong64.deb ...
正在解压 golang-go:loong64 (2:1.24-2) ...
正在选中未选择的软件包 libpkgconf3:loong64。
准备解压 .../4-libpkgconf3 1.8.1-4_loong64.deb ...
正在解压 libpkgconf3:loong64 (1.8.1-4) ...
正在选中未选择的软件包 pkgconf-bin。
准备解压 .../5-pkgconf-bin 1.8.1-4_loong64.deb ...
正在解压 pkgconf-bin (1.8.1-4) ...
正在选中未选择的软件包 pkgconf:loong64。
准备解压 .../6-pkgconf 1.8.1-4_loong64.deb ...
正在解压 pkgconf:loong64 (1.8.1-4) ...
```

```
sudo apt install golang-go
```

```
ls25@ls25-pc:~/下载$ go version
go version go1.24.4 linux/loong64
```

输入 `go version` 查看 go 版本号, 自此 go 安装完成。

3.10.2 脚本安装

下载安装脚本 `install_go.sh` 并上传至板卡。



`install_go.sh`

在脚本所在文件目录下执行以下操作:

```
chmod +x install_go.sh
./install_go.sh
```



3.11 如何安装 rust

3.11.1 手动安装

先安装 curl 命令: `sudo apt-get install curl`

通过 curl 安装 rust: `curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh`

```
ls25@ls25-pc:~/下载$ curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
info: downloading installer

Welcome to Rust!

This will download and install the official compiler for the Rust
programming language, and its package manager, Cargo.

Rustup metadata and toolchains will be installed into the Rustup
home directory, located at:

    /home/ls25/.rustup

This can be modified with the RUSTUP_HOME environment variable.

The Cargo home directory is located at:

    /home/ls25/.cargo

This can be modified with the CARGO_HOME environment variable.

The cargo, rustc, rustup and other commands will be added to
Cargo's bin directory, located at:

    /home/ls25/.cargo/bin

This path will then be added to your PATH environment variable by
modifying the profile files located at:

    /home/ls25/.profile
```

`source "/home/ls25/.cargo/env"`

`rustc -V`

```
20.4 MiB / 20.4 MiB (100 %) 7.1 MiB/s in 2s
info: installing component 'rust-std'
37.8 MiB / 37.8 MiB (100 %) 9.4 MiB/s in 4s
info: installing component 'rustc'
100.4 MiB / 100.4 MiB (100 %) 10.9 MiB/s in 9s
info: installing component 'rustfmt'
info: default toolchain set to 'stable-loongarch64-unknown-linux-gnu'

    stable-loongarch64-unknown-linux-gnu installed - rustc 1.91.1 (ed61e7d7e 2025-11-07)

Rust is installed now. Great!

To get started you may need to restart your current shell.
This would reload your PATH environment variable to include
Cargo's bin directory ($HOME/.cargo/bin).

To configure your current shell, you need to source
the corresponding env file under $HOME/.cargo.

This is usually done by running one of the following (note the leading DOT):
. "$HOME/.cargo/env" # For sh/bash/zsh/ash/dash/pdksh
source "$HOME/.cargo/env.fish" # For fish
source "$($nu.home-path)/.cargo/env.nu" # For nushell
ls25@ls25-pc:~/下载$ source ~/.cargo/env
ls25@ls25-pc:~/下载$ rustc -V
rustc 1.91.1 (ed61e7d7e 2025-11-07)
ls25@ls25-pc:~/下载$
```



3.11.2 脚本安装

下载安装脚本 `install_rust.sh` 并上传至板卡。



`install_rust.sh`

在脚本所在文件目录下执行以下操作：

```
chmod +x install_rust.sh
./install_rust.sh
```

3.12 如何安装 nodejs

3.12.1 手动安装

Nodejs 下载链接：<https://unofficial-builds.nodejs.org/download/release/>
选择 `loong64` 的版本。

← → ↺ 🏠 🔍 unofficial-builds.nodejs.org/download/release/v24.15.0/

Index of /download/release/v24.15.0/

../	16-Apr-2026 20:18	1248
SHASUMS256.txt	16-Apr-2026 07:22	9958714
node-v24.15.0-headers.tar.gz	16-Apr-2026 07:22	578440
node-v24.15.0-headers.tar.xz	16-Apr-2026 12:23	58594518
node-v24.15.0-linux-arm64-musl.tar.gz	16-Apr-2026 12:25	32106376
node-v24.15.0-linux-arm64-musl.tar.xz	16-Apr-2026 18:09	56564275
node-v24.15.0-linux-loong64.tar.gz	16-Apr-2026 18:11	30062304
node-v24.15.0-linux-loong64.tar.xz	16-Apr-2026 20:16	51916711
node-v24.15.0-linux-riscv64-pointer-compression..>	16-Apr-2026 20:18	27429316
node-v24.15.0-linux-riscv64-pointer-compression..>	16-Apr-2026 15:19	56208914
node-v24.15.0-linux-riscv64.tar.gz	16-Apr-2026 15:21	31199064
node-v24.15.0-linux-riscv64.tar.xz	16-Apr-2026 08:58	57743285
node-v24.15.0-linux-x64-musl.tar.gz	16-Apr-2026 09:00	31990008
node-v24.15.0-linux-x64-musl.tar.xz		

```
tar -xf node-v24.15.0-linux-loong64.tar.gz -C /opt/
echo export PATH=/opt/node-v24.15.0-linux-loong64/bin:$PATH >> ~/.bashrc
source ~/.bashrc
node -v
```

3.12.2 脚本安装

下载安装脚本 `install_node-js.sh` 并上传至板卡。



install_node-js.sh

在脚本所在文件目录下执行以下操作：

```
chmod +x install_node-js.sh
./install_node-js.sh
```

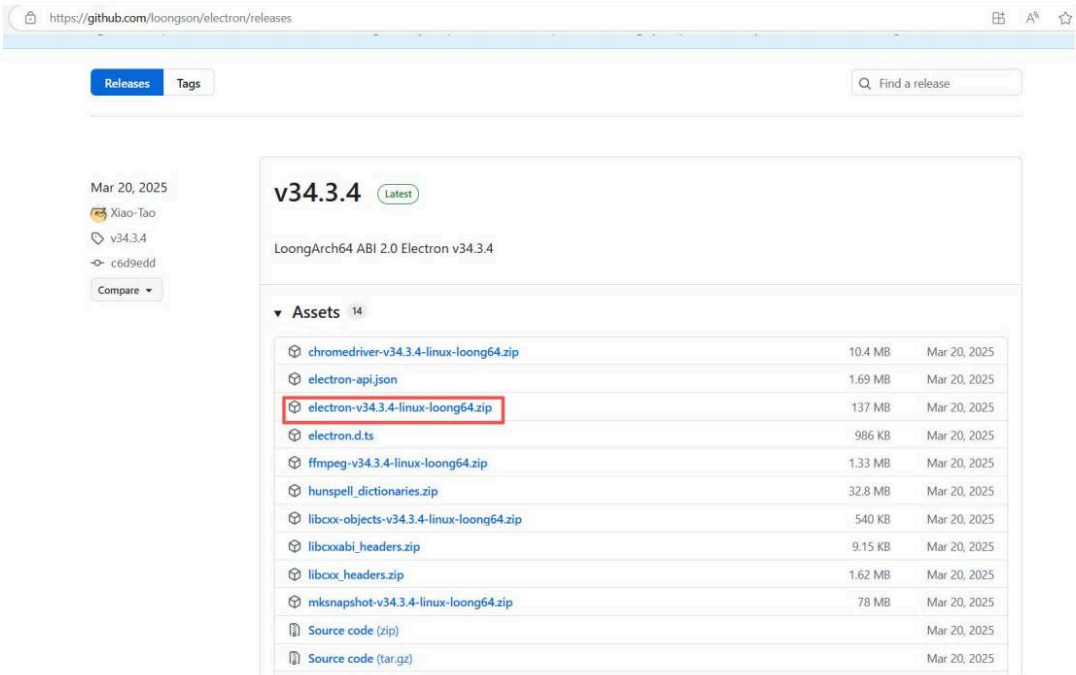
3.13 如何安装 electron

Electron 是一个使用 JavaScript、HTML 和 CSS 构建桌面应用程序的框架。通过将 Chromium 和 Node.js 嵌入到其二进制文件中，Electron 允许您维护一个 JavaScript 代码库并创建可在 Windows、macOS 和 Linux 上运行的跨平台应用程序——无需原生开发经验。

3.13.1 手动安装

下面以非全局安装使用 Electron 34.3.4 为例。（如果想要全局安装，可以直接参考开源社区中 Electron 安装说明文档，注意：全局安装可能会影响其他本地项目依赖。非必要情况，应避免使用管理员权限进行全局安装。）

安装包下载地址：<https://github.com/loongson/electron/releases>



新建一个目录（自定名称）

```
mkdir electron_test
cd electron_test
```



```

cp ~/下载/electron-v34.3.4-linux-loong64.zip ./
unzip electron-v34.3.4-linux-loong64.zip -d ./
./electron --version

ls@ls-pc:~/electron_test$ cp ~/下载/electron-v34.3.4-linux-loong64.zip ./
ls@ls-pc:~/electron_test$ unzip electron-v34.3.4-linux-loong64.zip -d ./
Archive:  electron-v34.3.4-linux-loong64.zip
  inflating: ./locales/lt.pak
  inflating: ./locales/it.pak
  inflating: ./locales/ta.pak
  inflating: ./libffmpeg.so
  inflating: ./locales/en-GB.pak
  inflating: ./resources.pak
  inflating: ./locales/da.pak
  inflating: ./locales/de.pak
  inflating: ./locales/et.pak
  inflating: ./locales/ar.pak
  inflating: ./locales/cs.pak
  inflating: ./locales/fil.pak
  inflating: ./locales/sr.pak
  inflating: ./libvk_swiftshader.so
  inflating: ./LICENSES.chromium.html
  inflating: ./libvulkan.so.1
  inflating: ./locales/ro.pak
  inflating: ./locales/zh-TW.pak
  inflating: ./locales/zh-CN.pak
  inflating: ./locales/hr.pak
  inflating: ./locales/tr.pak
  inflating: ./locales/ms.pak
  inflating: ./locales/th.pak

ls@ls-pc:~/electron_test$ chmod +x electron
ls@ls-pc:~/electron_test$ ./electron --version
v34.3.4
ls@ls-pc:~/electron_test$ ./electron

Electron 34.3.4 - Build cross platform desktop apps with JavaScript, HTML, and CSS
Usage: electron [options] [path]

A path to an Electron app may be specified. It must be one of the following:
- index.js file.
- Folder containing a package.json file.
- Folder containing an index.js file.
- .html/.htm file.
- http://, https://, or file:// URL.

Options:
-i, --interactive      Open a REPL to the main process.
-r, --require          Module to preload (option can be repeated).
-v, --version          Print the version.
-a, --abi              Print the Node ABI version.

```

查看版本号: `./electron --version`

弹出界面 `./electron`



3.13.2 脚本安装

下载安装脚本 `install_electron.sh` 并上传至板卡。



`install_electron.sh`

在脚本所在文件目录下执行以下操作：

```
chmod +x install_electron.sh
./install_electron.sh
```

3.13.3 Electron 使用示例

以一个简单使用示例程序为例，

`vim main.js` 添加以下内容：

```
const { app, BrowserWindow } = require('electron');

app.whenReady().then(() => {
  const win = new BrowserWindow({
    width: 800,
    height: 600
  });
  win.loadFile('index.html'); // 这里就是你的网页界面
});
```



```
loongson@loongson-pc:~$ cat main.js
const { app, BrowserWindow } = require('electron');

app.whenReady().then(() => {
  const win = new BrowserWindow({
    width: 800,
    height: 600
  });
  win.loadFile('index.html'); // 这里就是你的网页界面
});
```

vim index.html 添加以下内容:

```
<h1>龙芯 Electron 安装</h1>
<textarea></textarea>
```

```
loongson@loongson-pc:~$ cat index.html
<h1>龙芯 Electron安装</h1>
<textarea></textarea>
```

输入 `./electron main.js` 运行程序，成功输出界面。



3.14 如何安装.NET

3.14.1 手动安装

.NET 是一个免费的开源跨平台开发平台，支持多种语言（如 C#、F# 和 Visual Basic）和多



种应用类型，包括 Web 应用、桌面应用、移动应用、云原生应用、物联网（IoT）、游戏开发等。

dotnet-sdk 下载地址：<https://github.com/loongson/dotnet/releases>

v11.0.100-preview.3.26207.106-loongarch64 Pre-release Compare

github-actions released this 2 weeks ago · 8 commits to build since this release · v11.0.100-prev... · a8b33e7

Release .NET SDK 11.0.100-preview.3.26207.106 for LoongArch

The nupkgs with RID `linux-loongarch64` and `linux-musl-loongarch64` are provided by the source <https://nuget.loongnix.cn>. You can add this to your NuGet configuration using the following command:

```
dotnet nuget add source -n lnuget.loongnix.cn --protocol-version 3 https://lnuget.loongnix.cn/v3/index.json
```

Assets 6

dotnet-sdk-11.0.100-preview.3.26207.106-linux-loongarch64.tar.gz	sha256:a7e369cbda95d07caa36fedc65a16...	209 MB	2 weeks ago
dotnet-sdk-11.0.100-preview.3.26207.106-linux-musl-loongarch64.tar.gz	sha256:1df039fec62e4ce6baa8ae124ada9...	209 MB	2 weeks ago
Private.SourceBuilt.Artifacts.11.0.100-preview.3.26207.106.linux-loongarch64.tar.gz	sha256:544dbc6ebecb3c7bf7390544bd6e5...	438 MB	2 weeks ago
Private.SourceBuilt.Artifacts.11.0.100-preview.3.26207.106.linux-musl-loongarch64.t...	sha256:dc5ba1024177768e771629889c6ec...	440 MB	2 weeks ago
Source code (zip)			3 weeks ago
Source code (tar.gz)			3 weeks ago

默认是下载到“~/下载”文件夹中。执行以下命令安装并配置环境变量：

```
sudo mkdir -p /usr/share/dotnet

sudo tar -zxf dotnet-sdk-11.0.100-preview.3.26207.106-linux-loongarch64.tar.gz -C /usr/share/dotnet

sudo ln -sf /usr/share/dotnet/dotnet /usr/bin/dotnet

echo 'export DOTNET_ROOT=/usr/share/dotnet' | sudo tee -a /etc/profile

echo 'export PATH=$PATH:$DOTNET_ROOT' | sudo tee -a /etc/profile

source /etc/profile

dotnet --version
```

```
ls@ls-pc:~$ sudo mkdir -p /usr/share/dotnet
ls@ls-pc:~$ sudo tar -zxf ~/下载/dotnet-sdk-11.0.100-preview.3.26207.106-linux-loongarch64.tar.gz -C /usr/share/dotnet
ls@ls-pc:~$ sudo ln -sf /usr/share/dotnet/dotnet /usr/bin/dotnet
ls@ls-pc:~$ echo 'export DOTNET_ROOT=/usr/share/dotnet' | sudo tee -a /etc/profile
export DOTNET_ROOT=/usr/share/dotnet
ls@ls-pc:~$ echo 'export PATH=$PATH:$DOTNET_ROOT' | sudo tee -a /etc/profile
export PATH=$PATH:$DOTNET_ROOT
ls@ls-pc:~$ source /etc/profile
ls@ls-pc:~$ dotnet --version
11.0.100-preview.3.26207.106
ls@ls-pc:~$
```

测试能否正常使用：

```
dotnet new console -n myTestApp

cd myTestApp

dotnet run
```



```
ls@ls-pc:~$ dotnet new console -n MyTestApp
已成功创建模板“控制台应用”。

正在处理创建后操作...
正在还原 /home/ls/MyTestApp/MyTestApp.csproj:
已成功还原。

ls@ls-pc:~$ cd MyTestApp/
ls@ls-pc:~/MyTestApp$ dotnet run
Hello, World!
```

3.14.2 脚本安装

下载安装脚本 `install_dotnet10.sh` 并上传至板卡。



`install_dotnet10.sh`

在脚本所在文件目录下执行以下操作：

```
chmod +x install_dotnet10.sh
./install_dotnet10.sh
```

3.15 如何安装 jetbrains 软件

建议先根据 3.9 3.10 3.11 完成相关依赖软件安装，再安装 jet brain 相关软件，否则程序无法打开。

参考网址如下：<https://www.jetbrains.com/edu-products/download/?section=idea>

3.15.1 安装 IDEA

1.手动安装

访问网址：<https://www.jetbrains.com/edu-products/download/?section=idea>，下载版本 `idea-2026.1.1.tar.gz`，存放在“~/下载”文件夹中。



获取您的教育工具

Java Kotlin Python Scala JavaScript C/C++ Rust Go PHP



版本: 2025.2.5
内部版本号: 252.28238.7
2025年11月20日
[版本说明](#)

系统要求

安装说明

其他版本

使用 IntelliJ IDEA Community Edition 免费学习和教授 Java。

要开始在您的 IDE 中学习或教授 Java，请下载 IntelliJ IDEA Community Edition，切换到 *Learn*（学习）标签页，并点击 *Enable Access*（启用访问）。

下载 ^

Windows (.exe)

MacOS (.dmg, Apple Silicon)

MacOS (.dmg, Intel)

Linux (.tar.gz)

emy 创建真实世界应用程序

精进特定技能，例如使用 Spring Boot 开发框架开发桌面应用程序。

```
cd ~/下载
```

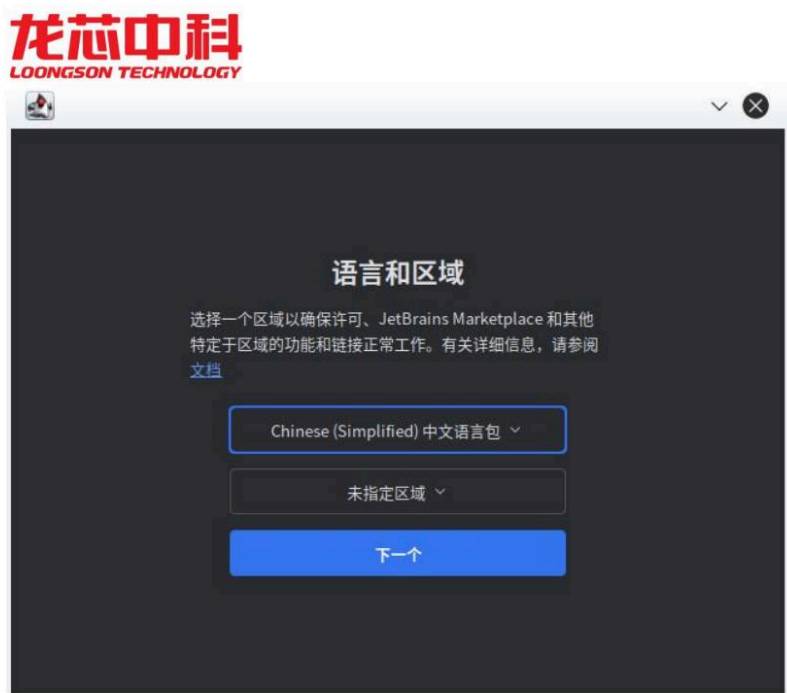
```
sudo tar -xf idea-2026.1.1.tar.gz -C /opt/
```

```
cd /opt/idea-IU-261.23567.138/bin/
```

```
./idea.sh
```

```
[root@pc-7805 ~]# ls
[ideal-2025-9.9.tar.gz] [longson2025-2.22-fcfd25-9.1-8-linux-longsonr604.deb]
[root@pc-7805 ~]# sudo tar -xzf ideal-2025-2.5.tar.gz -C /opt/
[root@pc-7805 ~]# cp opt/idea-IC-252.28328.7/lib/*
[root@pc-7805 ~]# cp opt/idea-IC-252.28328.7/bin/*
appletviewer.policy brokenPlugins.db format.sh fontchooser idea idead4 vmoptions idea.png idea.properties idea.sh idea.svg inspect.sh jetbrains_client64.vmoptions jetbrains_client.sh lteidit.sh restarter
[root@pc-7805 ~]# cp opt/idea-IC-252.28328.7/bin/* ./idea.sh
[0.00s][warning][cds] Archived non-system classes are disabled because the java.system.class.loader property is specified (value = "com.intelliJ.util.lang.PathClassLoader"). To use archived non-system classes, this property must not be set
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by com.sun.jna.Native in an unnamed module
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

2025-11-25 10:44:15.388 [ 75] WARN - 8-c.i.p.s.AppStarter - Unable to load JNA library (linux/x86_64) [opt/idea-IC-252.28328.7/lib/jna/jnaamd64]
java.Lang.UnsatisfiedLinkError: Unable to locate JNA native support library
    at com.sun.jna.Native.loadNativeDispatchLibrary(Native.java:1014)
    at com.sun.jna.Native.<init>(Native.java:221)
    at com.intelliJ.jna.JnaLoader.load(StdLoader.java:29)
    at com.intelliJ.platform.ide.bootstrap.StartupUtil$ScheduleLoadSystemClassAndInfoInInitMacApp$112.invokeSuspend(Startup.kt:353)
    at kotlin.coroutines.jvm.internal.BaseContinuationImpl.resumeWith(ContinuationImpl.kt:33)
    at kotlinx.coroutines.DispatchedTask.run(DispatchedTask.kt:100)
    at kotlinx.coroutines.InternalSoftInterruptDispatcher$Marker.run(SoftInterruptDispatcher.kt:130)
    at kotlinx.coroutines.scheduler.TaskImpl.run(Tasks.kt:40)
    at kotlinx.coroutines.scheduler.CoroRuntimeScheduler_runsafely(CoroutineScheduler.kt:613)
    at kotlinx.coroutines.scheduler.CoroRuntimeScheduler$Marker.executeTask(CoroutineScheduler.kt:1183)
    at kotlinx.coroutines.scheduler.CoroRuntimeScheduler$Marker.runMarker(CoroutineScheduler.kt:778)
    at kotlinx.coroutines.scheduler.CoroRuntimeScheduler$Marker.run(CoroutineScheduler.kt:765)
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::invokeCleaner has been called by com.intelliJ.util.lang.ByteBufferCleaner (file:/opt/idea-IC-252.28328.7/lib/platform-loader.jar)
```



按照提示点击即可。

打开软件后，选中左上角工具栏的 `tool` -> 创建桌面条目，可在菜单找到该软件，接着右键“添加到桌面”（“create desktop entry”），后续即可通过桌面快捷方式打开。

2. 脚本安装

访问网址：<https://www.jetbrains.com/edu-products/download/?section=idea>，下载当前版本，存放在“~/下载”文件夹中。

下载安装脚本 `install_idea.sh` 并上传至板卡。



`install_idea.sh`

在脚本所在文件目录下执行以下操作：

```
chmod +x install_idea.sh
./install_idea.sh
```

3.15.2 安装 Goland

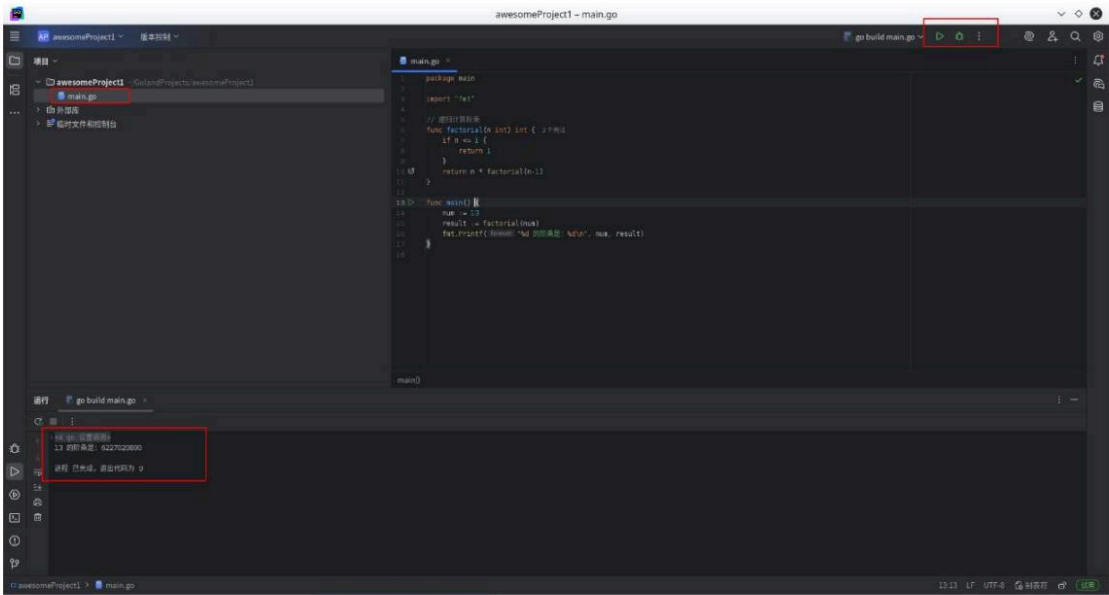
1. 手动安装

访问网址：<https://www.jetbrains.com/edu-products/download/?section=goland-go>，下载版本 `Goland-2026.1.tar.gz`，存放在“~/下载”文件夹中。

```
cd ~/下载
sudo tar -xf goland-2026.1.tar.gz -C /opt/
cd /opt/GoLand-2026.1/bin
./goland.sh
```



以下是安装完成后运行图：



2. 脚本安装

访问网址：<https://www.jetbrains.com/edu-products/download/?section=goland-go>，下载当前版本，存放在“~/下载”文件夹中。

下载安装脚本 `install_goland.sh` 并上传至板卡。



`install_goland.sh`

在脚本所在文件目录下执行以下操作：

```
chmod +x install_goland.sh
./install_goland.sh
```

接下来添加 debug 调试功能：（原本的软件 debug 依赖 x86 库，无法使用，因此需要替换 `dlv` 可执行文件。）

`dlv` 文件有两种方法获取，

一：从网盘获取：文件系统->Goland-delve，之后直接跳到步骤（4）； 二：自行编译。

自行编译 `dlv` 步骤如下：

- (1) 从 `dlv` 获取上游源码：`git clone https://github.com/go-delve/delve.git`
- (2) 编译：`go build -tags=exp.linuxloong64 -o dlv cmd/dlv/main.go` 得到可执行文件 `dlv`，至此，编译完成。

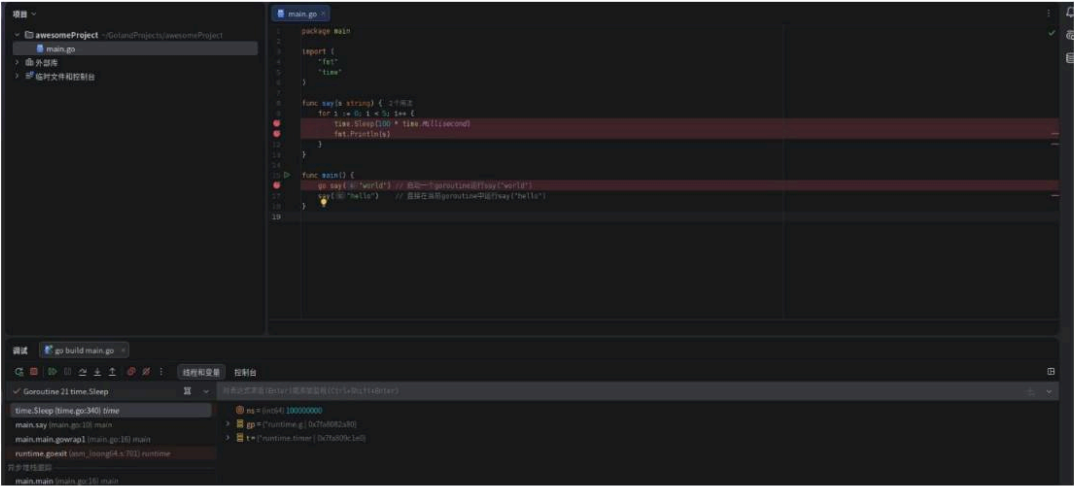
```
ls@ls-pc:~/下载/delve-1.25.2$ ls
assets  CHANGELOG.md  cmd  CONTRIBUTING.md  Documentation  _fixtures  go.mod  go.sum  ISSUE_TEMPLATE.md  LICENSE  Makefile  pkg  README.md  _scripts  service  vendor
ls@ls-pc:~/下载/delve-1.25.2$ go build -tags=exp.linuxloong64 -o dlv cmd/dlv/main.go
ls@ls-pc:~/下载/delve-1.25.2$ ls
assets  CHANGELOG.md  cmd  CONTRIBUTING.md  dlv  Documentation  _fixtures  go.mod  go.sum  ISSUE_TEMPLATE.md  LICENSE  Makefile  pkg  README.md  _scripts  service  vendor
```

(3) 接着 `dlv` 替换到如下对应目录（`/opt/GoLand-2025.3/plugins/go-plugin/lib/dlv/linux`）即



```
ls@ls-pc:/opt/GoLand-2025.3/plugins/go-plugin/lib/dlv/linux$ ls
dlv
ls@ls-pc:/opt/GoLand-2025.3/plugins/go-plugin/lib/dlv/linux$ pwd
/opt/GoLand-2025.3/plugins/go-plugin/lib/dlv/linux
ls@ls-pc:/opt/GoLand-2025.3/plugins/go-plugin/lib/dlv/linux$ sudo cp ~/dlv ./ -f
ls@ls-pc:/opt/GoLand-2025.3/plugins/go-plugin/lib/dlv/linux$ file dlv
dlv: ELF 64-bit LSB executable, LoongArch-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld.so.1, BuildID[sha1]=c4a568b05215abf77adc396ab0288e0fb33f3feb, with debug_info, not stripped
```

debug 正常使用演示图如下：



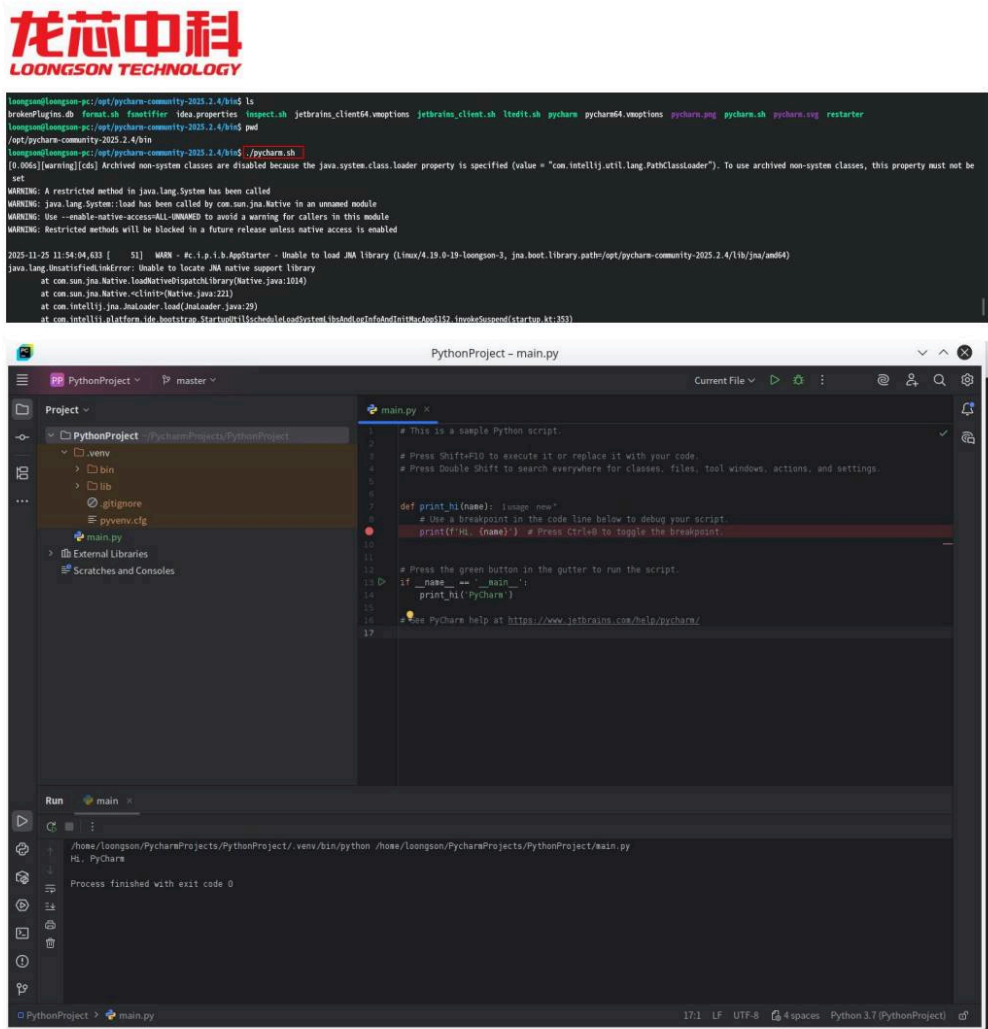
3.15.3 安装 pycharm

1.手动安装

访问网址：<https://www.jetbrains.com/edu-products/download/?section=pycharm-edu>，下载版本 pycharm-2025.3.tar.gz，存放在“~/下载”文件夹中。

```
cd ~/下载
sudo tar -xf pycharm-2025.3.tar.gz -C /opt/
cd /opt/pycharm-2025.3/bin
./pycharm.sh
```

```
ls@ls-pc:/opt/GoLand-2025.3/plugins/go-plugin/lib/dlv/linux$ ls
dlv
ls@ls-pc:/opt/GoLand-2025.3/plugins/go-plugin/lib/dlv/linux$ pwd
/opt/GoLand-2025.3/plugins/go-plugin/lib/dlv/linux
ls@ls-pc:/opt/GoLand-2025.3/plugins/go-plugin/lib/dlv/linux$ sudo cp ~/dlv ./ -f
ls@ls-pc:/opt/GoLand-2025.3/plugins/go-plugin/lib/dlv/linux$ file dlv
dlv: ELF 64-bit LSB executable, LoongArch-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld.so.1, BuildID[sha1]=c4a568b05215abf77adc396ab0288e0fb33f3feb, with debug_info, not stripped
```



2. 脚本安装

访问网址：<https://www.jetbrains.com/edu-products/download/?section=pycharm-edu>，下载当前版本，存放在“~/下载”文件夹中。

下载安装脚本 install_pycharm.sh 并上传至板卡。


install_pycharm.sh

在脚本所在文件目录下执行以下操作：

```

chmod +x install_pycharm.sh
./install_pycharm.sh

```

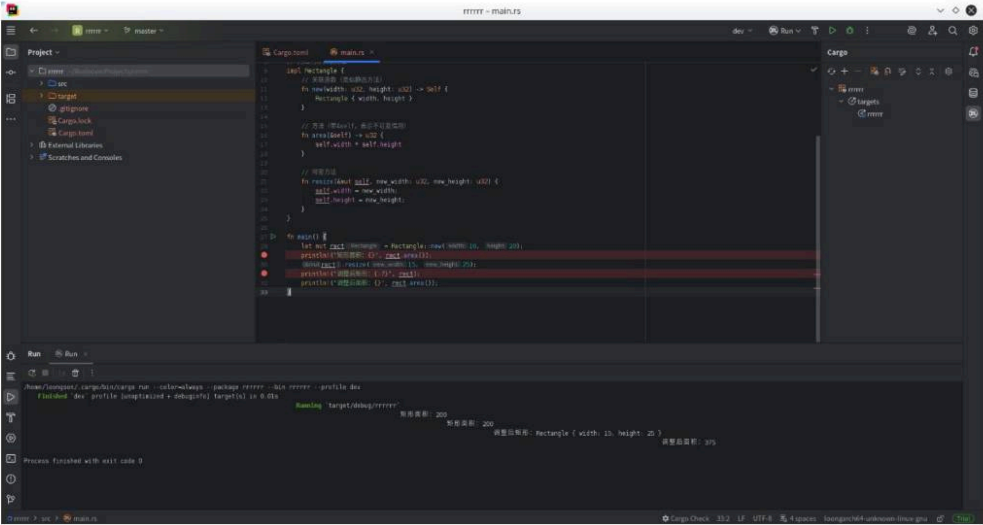
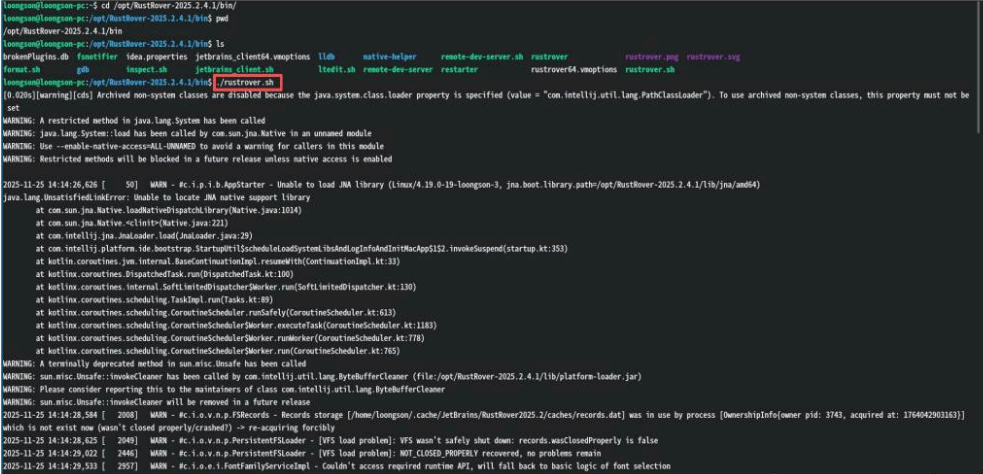
3.15.4 安装 Rustover

1.手动安装



访问网址：<https://www.jetbrains.com/edu-products/download/?section=rustrover>，下载版本 RustRover-2026.1.2.tar.gz，存放在“~/下载”文件夹中。

```
cd ~/下载
sudo tar -xf RustRover-2026.1.2.tar.gz -C /opt/
cd /opt/RustRover-2026.1.2/bin
./rustrover.sh
```



2. 脚本安装

访问网址：<https://www.jetbrains.com/edu-products/download/?section=pycharm-edu>，下载当前版本，存放在“~/下载”文件夹中。

下载安装脚本 install_rustrover.sh 并上传至板卡。



install_rustrover.sh



在脚本所在文件目录下执行以下操作：

```
chmod +x install_rustrover.sh  
./install_rustrover.sh
```

其他软件同理，不再一一演示。

3.16 如何安装 AI 代码工具

以下 AI 代码工具均依赖于 nodejs，请先参考 [3.12 如何安装 nodejs](#)。注意如果是安装 **copilot cli**，要求 nodejs 版本大于 24，可选择 24.15.0。

接着更新 npm 版本

```
npm install -g npm@11.13.0
```

3.16.1 安装 qwen-code

打开终端，依次输入以下命令即可：

```
npm install -g @qwen-code/qwen-code@latest
```

安装完成后，输入 `qwen -v` 查看版本信息。

打开任意代码项目，输入 `qwen` 打开命令行交互窗口，第一次打开时需要选择对应的认证方式登录。

具体使用可参考官方使用文档：<https://qwenlm.github.io/qwen-code-docs/zh/users/overview/>

3.16.2 安装 codebuddy

打开终端，输入以下命令即可：

```
npm install -g npm@11.13.0
```

```
npm install -g @tencent-ai/codebuddy-code
```

安装完成后，输入 `codebuddy --version` 查看版本信息。

具体使用可参考官方使用文档：<https://www.codebuddy.ai/docs/zh/cli/quickstart>

3.16.3 安装 copilot cli

打开终端，输入以下命令即可：

```
npm install -g @github/copilot
```

```
npm install node-pty
```

安装完成后，输入 `copilot--v` 查看版本信息。

具体使用可参考官方使用文档：<https://docs.github.com/zh/copilot/how-tos/copilot-cli/>

3.17 其他常用软件

下载链接：<https://github.com/loong64>



Binary applications.

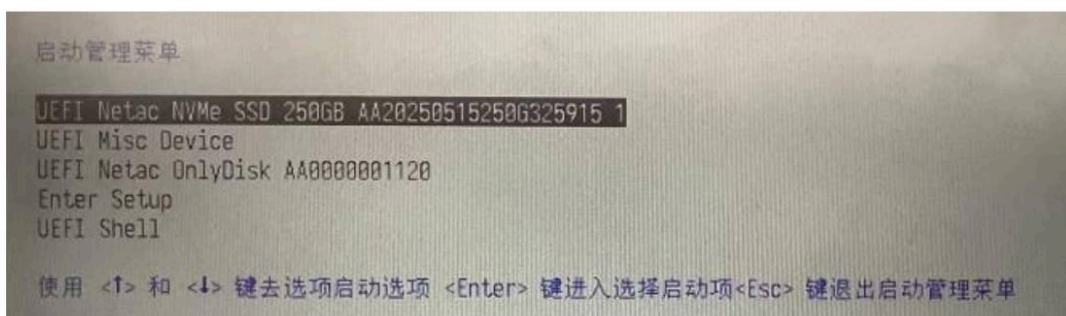
Name	Release	Description
gn	release v2024.12	Standalone version of Chromium's GN.
uv	release v0.11.5	An extremely fast Python package and project manager, written in Rust.
gosu	release v1.19	Simple Go-based setuid+setgid+setgroups+exec.
binfmt	release deploy/v10.2.1-23	Cross-platform emulator collection distributed with Docker images.
buildx	release v0.33.0	Docker CLI plugin for extended build capabilities with BuildKit.
compose	release v5.1.1	Define and run multi-container applications with Docker.
ossutil	release v1.7.19	A user friendly command line tool to access AliCloud OSS.
plugins	release v1.9.1	Some reference and example networking plugins, maintained by the CNI team.
llama.cpp	release b8709	LLM inference in C/C++.
cross-tools	release 20260403	LoongArch64 cross-compile toolchain, supports both x86_64(amd64) and aarch64(arm64) architectures.
cibuildwheel	release v3.4.1	 Build Python wheels for all the platforms with minimal configuration.
node-sqlite3	release v6.0.1	SQLite3 bindings for Node.js.
Box64	release v0.4.0	Linux Userspace x86_64 Emulator with a twist.
Ccache	release v4.13.2	Ccache is a compiler cache.
Cosign	release v3.0.6	Code signing and transparency for containers and binaries.
CMake	release v4.3.1	CMake is a cross-platform, open-source build system generator.
Docker	release v29.4.0	Packaging scripts for Docker CE
Godot	release v4.6.2-stable	Godot Engine – Multi-platform 2D and 3D game engine
Ollama	release v0.20.4	Get up and running with large language models.
Maturin	release v1.12.6	Build and publish crates with pyo3, cffi and uniffi bindings as well as rust binaries as python packages.

3. 18 系统启动选择

板卡支持从 EMMC、SSD 和 SATA 启动，若均安装有系统，可以选择其中任意一个作为启动项。

3. 18. 1 单次切换从 EMMC/SSD/SATA 启动系统

开发板开机，按 F12 后进入 UEFI 配置界面，选择 “启动管理” 进入启动菜单。

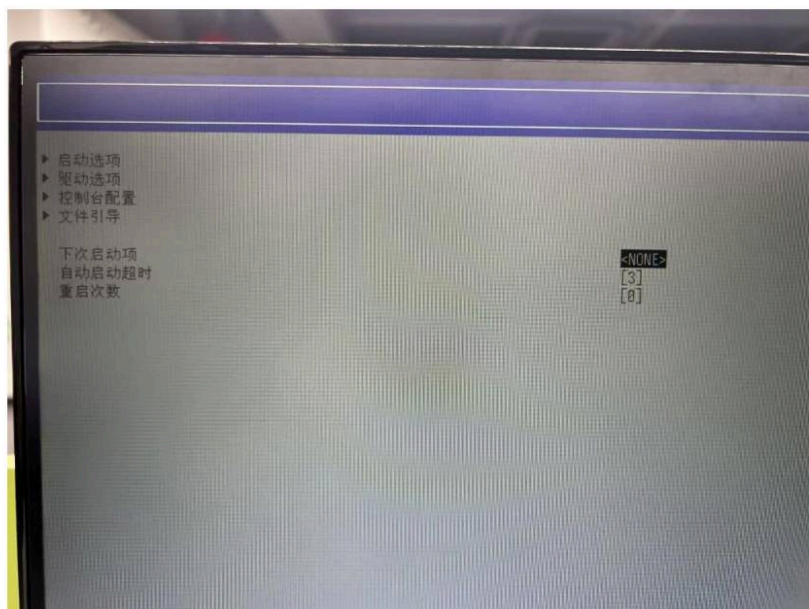
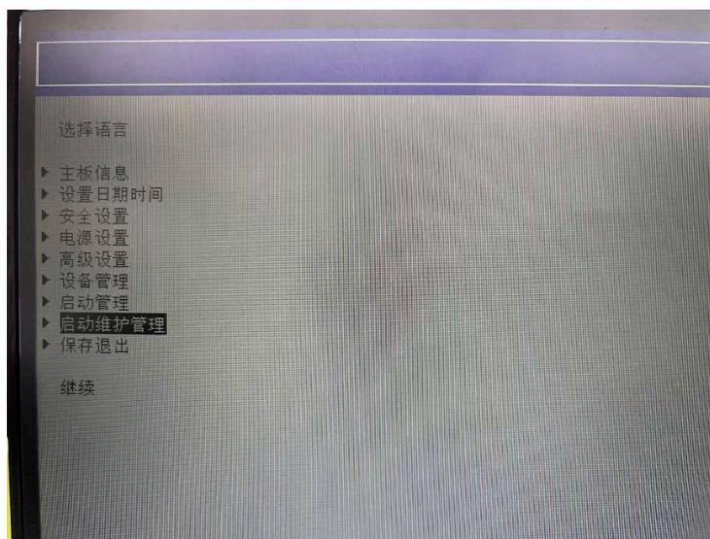


按方向键上下控制选中项，Enter 回车确认。如系统安装在 UEFI Netac NVMe SSD 250GB***，选中后，按回车启动系统。如果要通过 EMMC 启动系统，则选择 UEFI Misc Device。

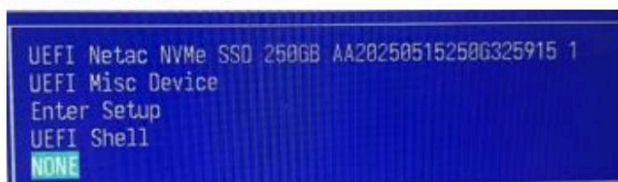
3.18.2 永久设置从 EMMC 或 SSD 启动系统

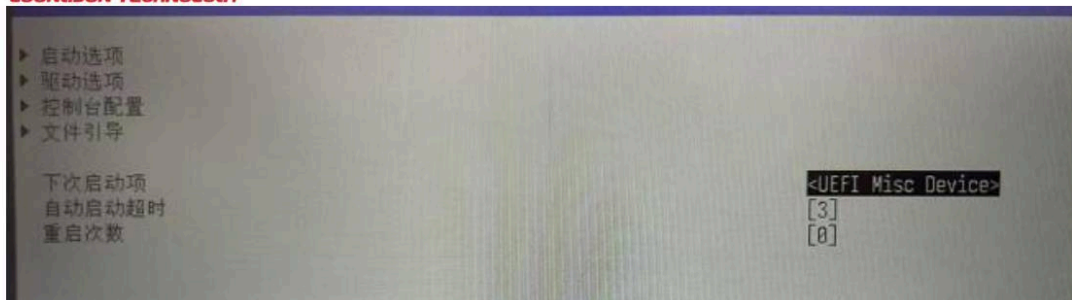
设置默认启动选项：设置完成后下次系统则默认走这个启动路径，不需要每次都手动选择 EMMC/SSD 启动系统。

选择“启动维护管理”选项回车



找到“下次启动项”，回车，默认使用 MMC 启动，则选中 UEFI Misc Device。默认使用 SSD 启动，则选中 UEFI Netac NVMe SSD 256GB ***。





接下来按 F10，保存退出重启。

四、系统安装

4.1 制作系统盘

首先下载龙芯引导系统镜像 25（以最新版为准）

系统镜像可以从 <https://www.loongnix.cn/zh/loongnix/> 下载，选择 Loongnix-25 的版本即可。

接下来分别演示 windows 和 Linux 平台（使用龙芯架构的机器参考 linux 平台即可）制作方法

4.1.1 windows 平台

UltraISO 以管理员身份打开，U 盘启动盘制作如下步骤：



特别注意：U 盘格式需为 FAT32， 写入方式必须改为 RAW。建议先对 U 盘执行格式化之后再 写入。

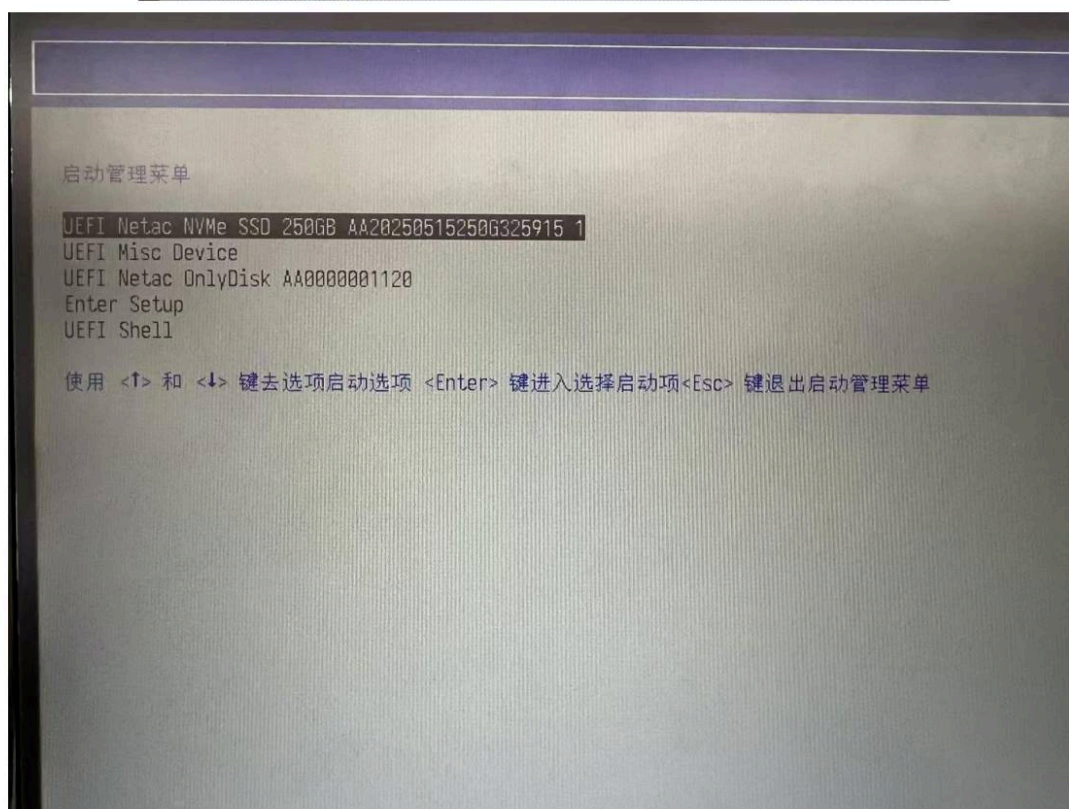
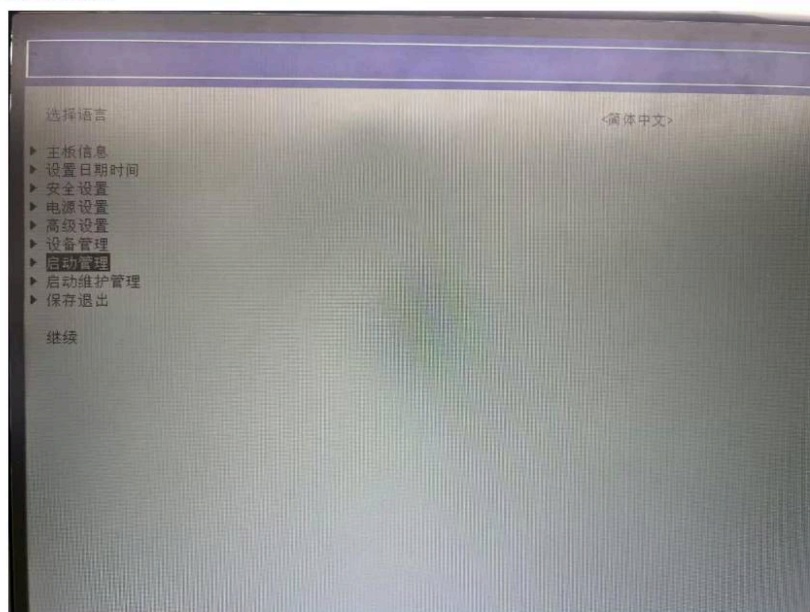
4.1.2 Linux 平台

sudo dd if=xxx.iso of=/dev/sdx xxx.iso 为镜像名称， /dev/sdx 是 U 盘的设备节点名，可通过 fdisk-l 命令查看。

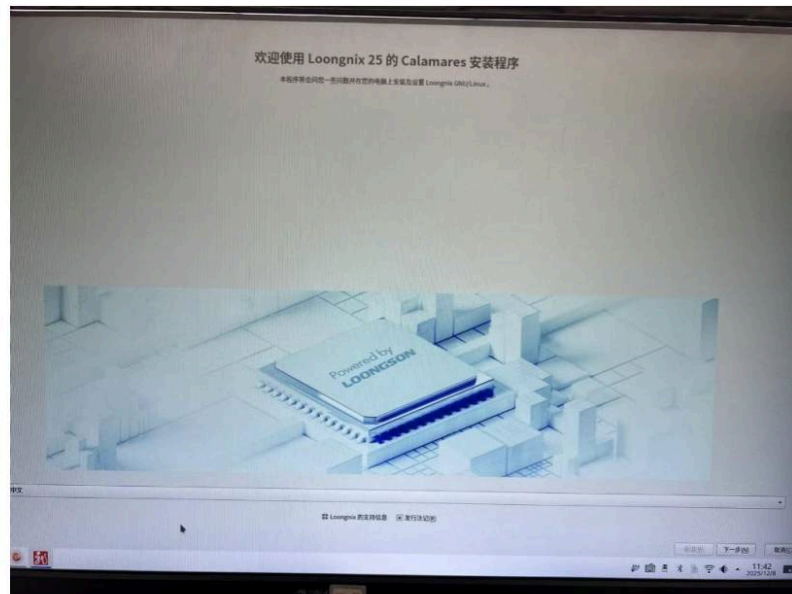
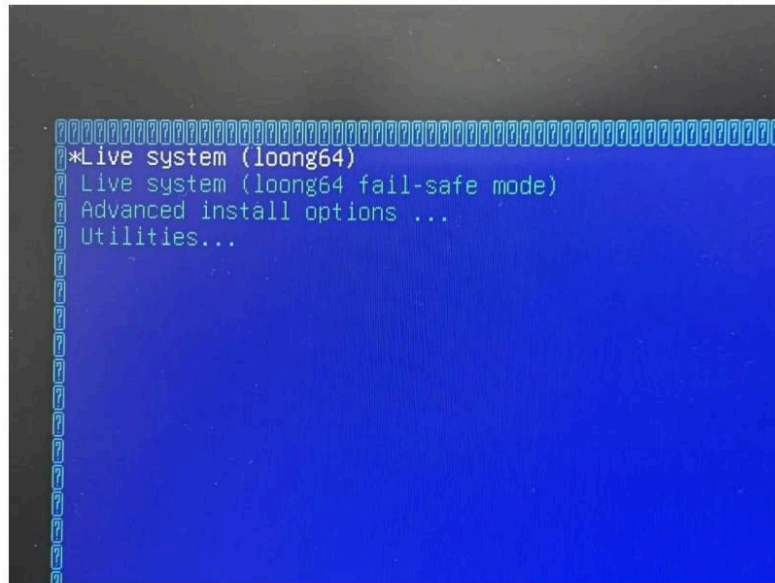
```
oo@oo-virtual-machine:~/Loongson/work$ sudo dd if=Loongnix-20.7.rc2.lived.kde.l
oongarch64.1030.iso of=/dev/sdb
[sudo] oo 的密码:
记录了5382756+0 的读入
记录了5382756+0 的写出
2755971072字节 (2.8 GB, 2.6 GiB) 已复制, 146.891 s, 18.8 MB/s
```

4.2 安装到 EMMC 或 SSD

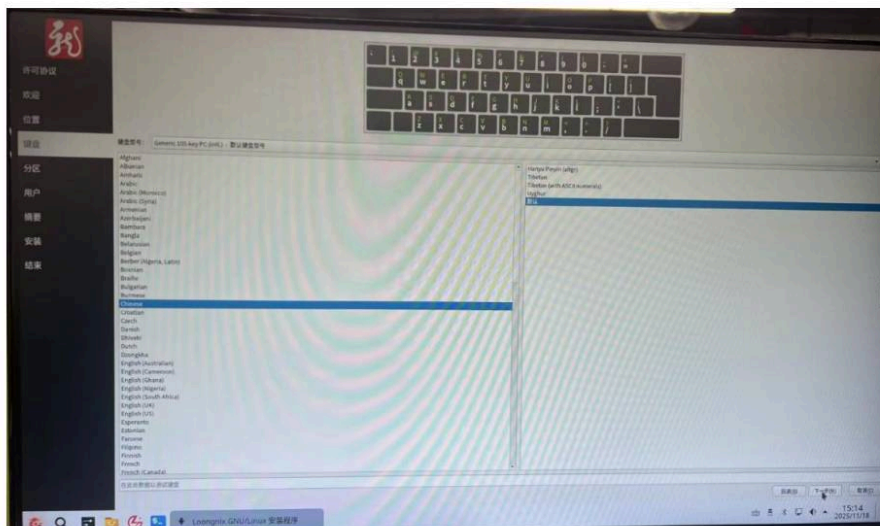
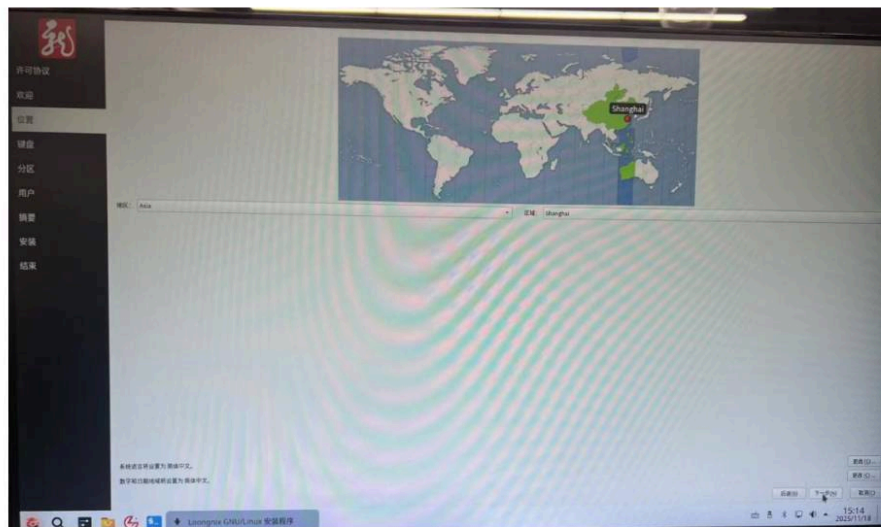
开发板开机，按 F2 后进入 UEFI 配置界面，选择 “启动管理” 进入启动菜单。



选中第三个（系统启动 U 盘为 OnlyDisk**，根据自己 U 盘名称实际选择），然后按回车。
进入 loongnix25 系统安装界面后，按照下图步骤点击即可。



点击下一步



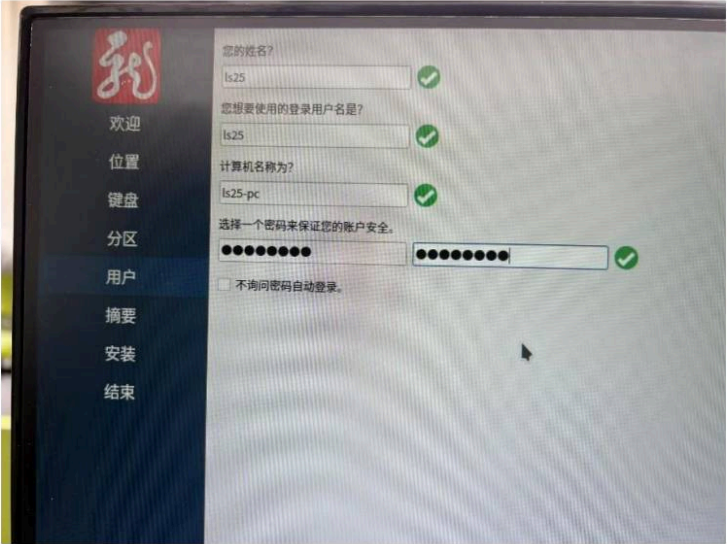
这里选择安装位置：如果想要安装到 EMMC 则选择上面 MMC (/dev/mmcblk0)，要安装到 SSD 则选择下面 SSD 选项。





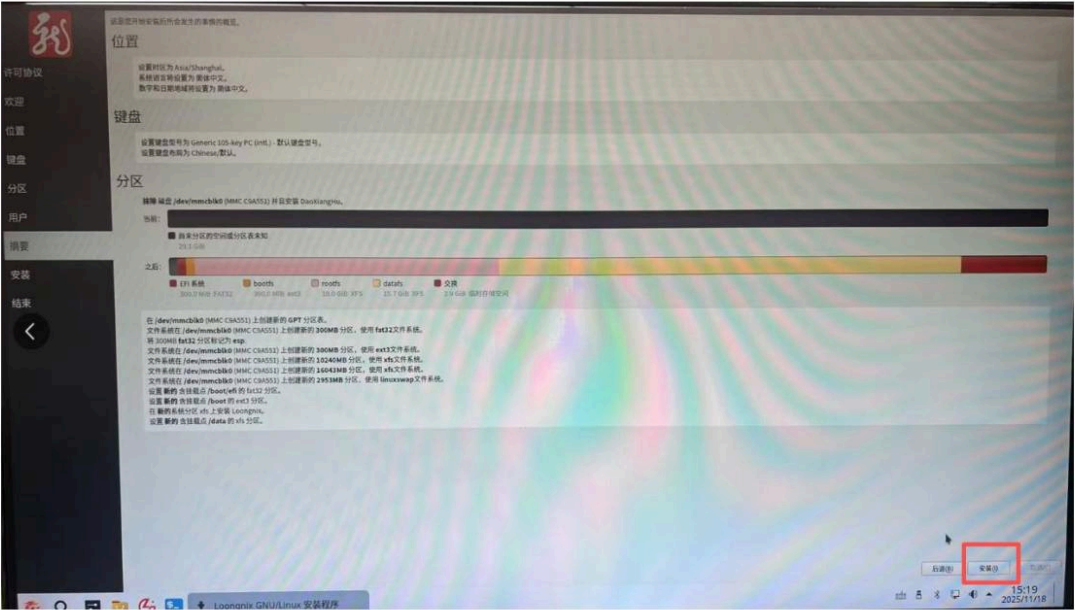
默认选择“抹除磁盘”选项，会自动分配分区。当然也可以选择手动分区，点击手动分区后，会有对应分区分配指引，按照要求选择即可。注意：手动分区时，务必确保严格按照指引，否则系统无法成功安装。

以下使用“抹除磁盘”选项，点击”下一步“ -> 点击“安装“。



注意：密码必须不少于 8 位，并且不能为简单的数字或字母。

等待安装完成重启系统即可。





五、固件更新

UEFI 获取路径：BSP包-->固件

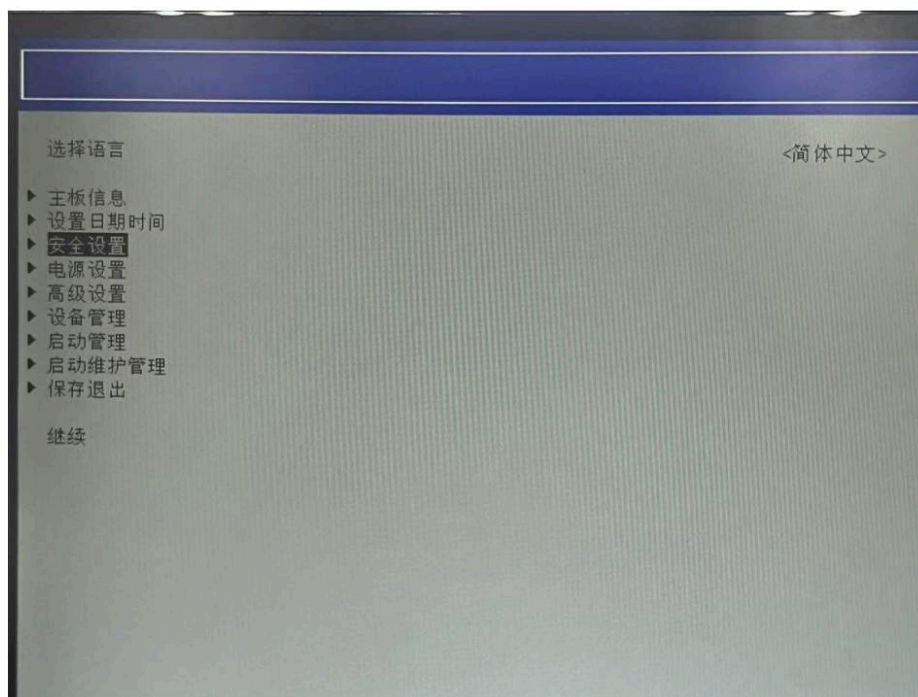
根据机型选择相应的固件（Image 目录有相应机型的实物照片）。



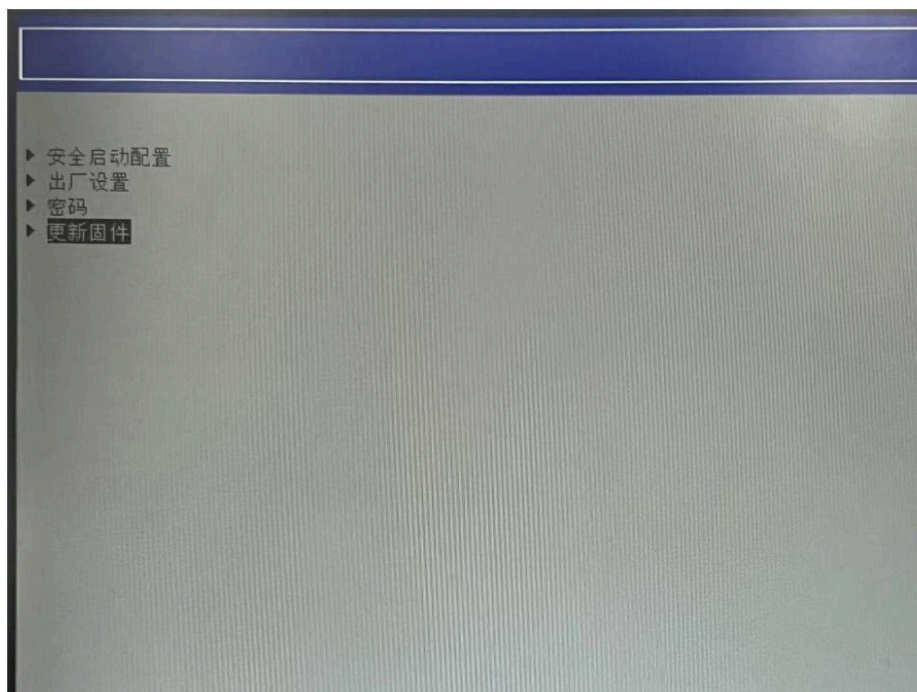
插好装有固件的U 盘（U盘格式为fat32或ext3格式）， 上电开机后待出现 loongson Logo 时按 F2键 进去UEFI配置界面。



5.1 方法一：通过图形化界面升级固件

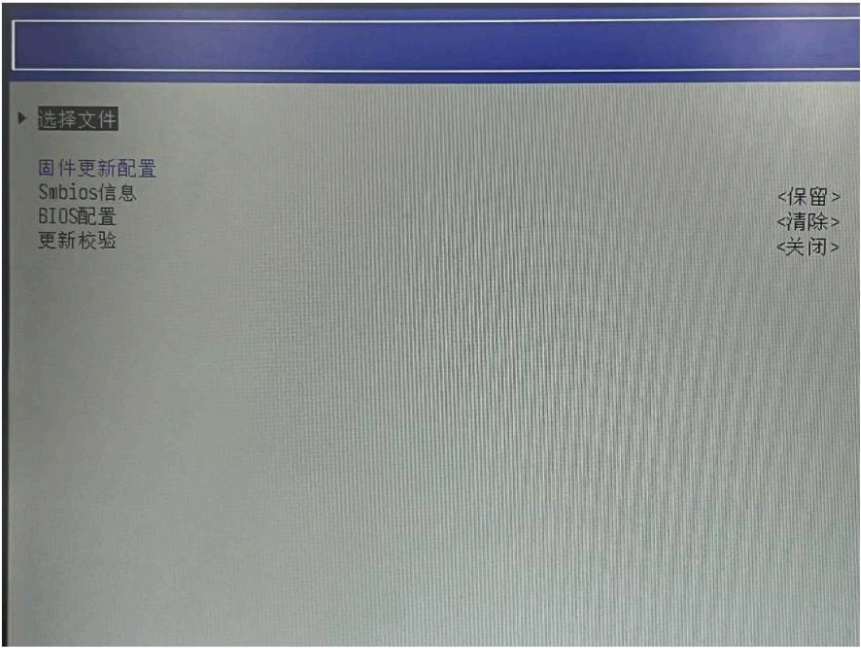


进入配置界面后通过上下选择键选中 **安全设置** 按回车键进入 **安全设置**

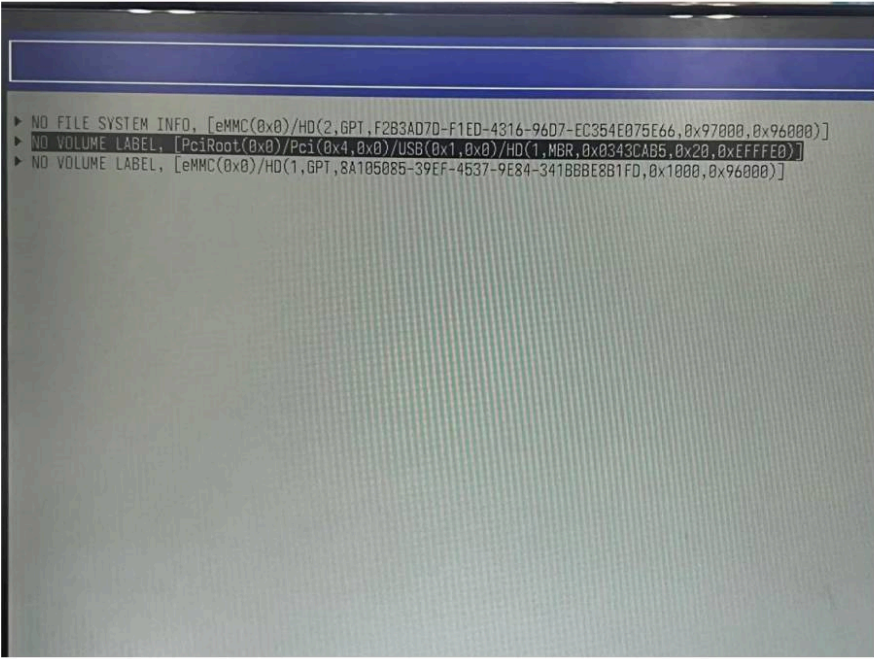




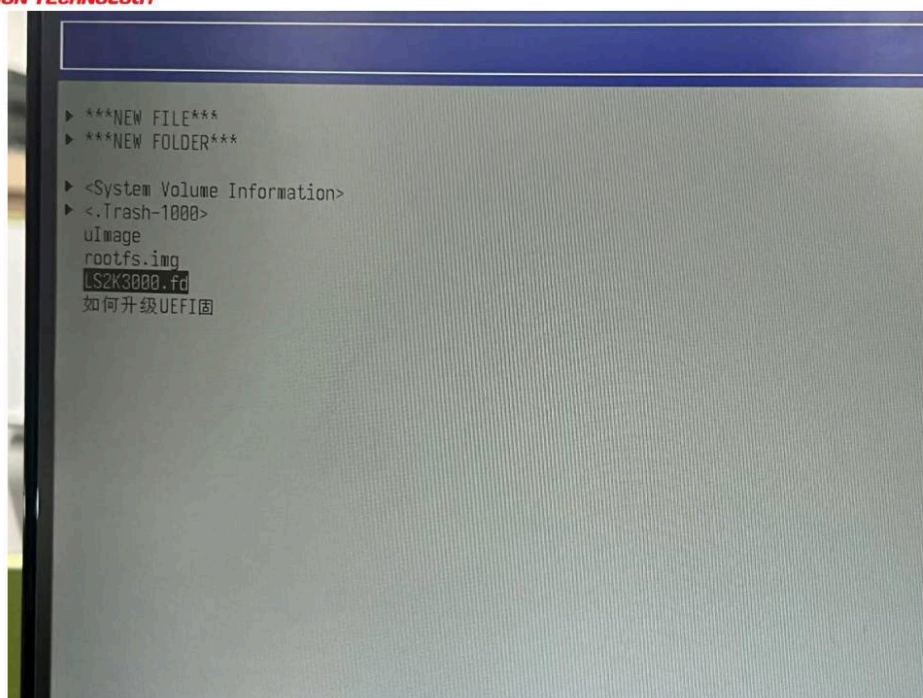
进入配置界面后通过上下选择键选中 **更新固件**，按回车键进入 **更新固件**



进入配置界面后通过上下选择键选中 **选择文件**按回车键进入选择文件



选择带有 **USB** 的即为 U 盘，如上图第二行，按回车键进入下一步。



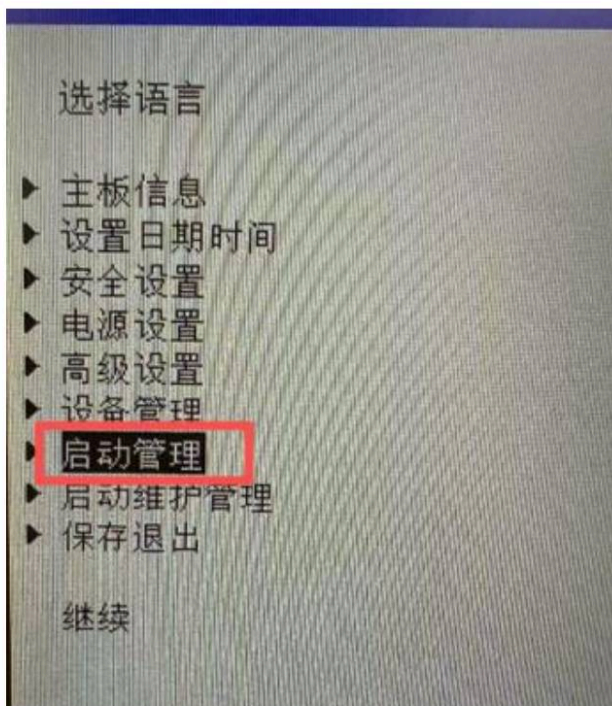
进入配置界面后通过上下选择键选中 LS2K3000. fd，按回车键确认更新固件。



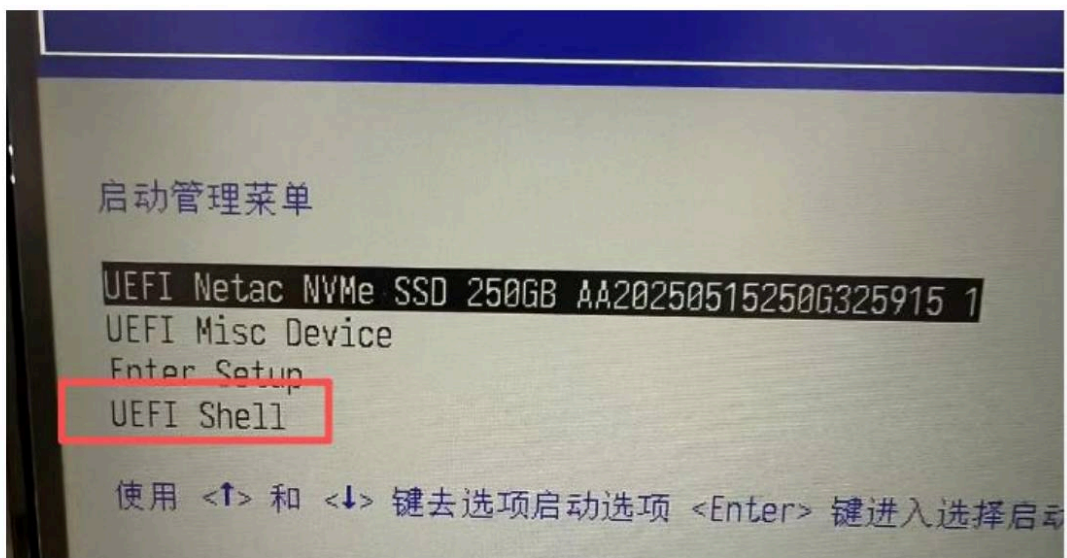
升级固件完成后重启系统。



5.2 方法二：通过命令行升级固件



进入配置界面后通过上下选择键选中 **启动管理**按回车键进入 **启动管理**



进入 **启动管理** 后选择 **UEFI Shell** 按回车进入UEFI 命令行。



```

UEFI Interactive Shell v2.2
EDK II
UEFI v2.70 (EDK II, 0x00010000)
Mapping table
FS1: Alias(s):HD2b::BLK4:
PciRoot(0x0)/Pci(0x9,0x0)/Pci(0x0,0x0)/NVMe(0x1,15-59-32-00-25-D5-82-0C)/HD(1,GPT,85EF0891-7DE4-4C1B-A904-120
FS2: Alias(s):HD2c::BLK5:
PciRoot(0x0)/Pci(0x9,0x0)/Pci(0x0,0x0)/NVMe(0x1,15-59-32-00-25-D5-82-0C)/HD(2,GPT,11856A1F-AC56-43E1-832E-549
FS0: Alias(s):HD1b0b::BLK2:
PciRoot(0x0)/Pci(0x4,0x0) USB(0x1,0x0) HD(1,GPT,798FD1F6-E84D-45C0-813D-93E1B52FEA10,0x800,0xFF000)
BLK0: Alias(s):
eMMC(0x0)
BLK3: Alias(s):
PciRoot(0x0)/Pci(0x9,0x0)/Pci(0x0,0x0)/NVMe(0x1,15-59-32-00-25-D5-82-0C)
BLK6: Alias(s):
PciRoot(0x0)/Pci(0x9,0x0)/Pci(0x0,0x0)/NVMe(0x1,15-59-32-00-25-D5-82-0C)/HD(3,GPT,49C5CA5A-2AAD-4726-8BF1-7580
BLK7: Alias(s):
PciRoot(0x0)/Pci(0x9,0x0)/Pci(0x0,0x0)/NVMe(0x1,15-59-32-00-25-D5-82-0C)/HD(4,GPT,FAB59D38-4B85-42F7-8F2B-285E
BLK8: Alias(s):
PciRoot(0x0)/Pci(0x9,0x0)/Pci(0x0,0x0)/NVMe(0x1,15-59-32-00-25-D5-82-0C)/HD(5,GPT,287DF25D-2CB2-49E0-A41D-AC8A0
BLK9: Alias(s):
PciRoot(0x0)/Pci(0x9,0x0)/Pci(0x0,0x0)/NVMe(0x1,15-59-32-00-25-D5-82-0C)/HD(6,GPT,781E3C12-EE15-4B8D-A497-711F9
BLK1: Alias(s):
PciRoot(0x0)/Pci(0x4,0x0)/USB(0x1,0x0)
Press F8 in 4 seconds to skip startup.nsh or any other key to continue.
Shell> FS0:
FS0:\> ls
Directory of: FS0:\
11/19/2025 09:42 <DIR> 4,096 Fresh-1000
09/01/2025 11:09 8,388,608 LS2K3000.fd
1 File(s) 8,388,608 bytes
1 Dir(s)
FS0:\>

```

按 **ESC** 键进入 shell，之后再查找 U 盘对应的 Mapping table，如上图所示 USB 对应的是 **FS0**：

```

FS0:\> help spi
Read or write spi flash.

SPI [-r StartAddr Size]/[-w data/-w StartAddr data]/[-u file]/[-e]/[-l file StartAddr Size]

-r          - read ls7a spi from address.
-w          - write ls7a spi data.
-u          - update ls3a spi flash.
-e          - erase ls7a spi flash by Sector.
-x          - download correct firmware to Serial ROM for xhci.
-l          - write ls7a spi data from start address to end address

NOTES:
1. This command can update flash you should be careful.

* update the flash:
fs0:\> spi -u uefi

* write the ls7a spi flash:
fs0:\> spi -w 00:11:22:33:44:55
fs0:\> spi -w 0x10 aa:bb:cc:dd:ee:ff

* write the ls7a spi flash:
fs0:\> spi -l vbios.bin 0x1000 0x20000

```

输入对应的标号，（本例中 U 盘的标号是 **FS0**：）进入 U 盘，输入 **help spi** 查看 SPI 命令的使用说明



```
FSB:> spi -i v0105.bin 0x1000 0x20000
FSB:> spi -u LS2K3000.fd
Now program spi flash, wait a while..
Erase : *****
*****
***** Erase OK.
*****
*****
Program : *****
*****
***** Program OK.
*****
*****
FSB:> vers
Version: Loongson-WDK2018-V4.0.05878-stable202411-f260b4b
Build time: 09/01/25 11:08:12
Commit: commit f260b4bcb7fb2a31bb531bd422fc521fd0d08c Author: tao89959 <1036524114@qq.com> Date: Fri Aug 29 17:58:16 2025 +0800
FSB:>
```

输入 `spi -u xxx.fd` 升级固件，固件升级过程为先擦除，后写入，待出现 `Program OK` 后，表示升级完成，输入 `reset` 重启电脑即可。

输入 **vers** 可查看固件的版本信息，版本信息包括版本号，编译时间，git commit ID。

PS: 更新固件后第一次重启需要等几分钟才有 logo 输出, 因为有内存训练的过程。